Belegarbeit

zur Lehrveranstaltung
Echtzeitsysteme und mobile Robotik

Sommersemester 2026

Dozenten: M. Sc. Moritz Thümmler, M. Sc. Nico Beyer

vorgelegt von:	Lennart Mende
Studiengang:	Elektrotechnik und Informationstechnik
Matrikelnummer:	81256
E-Mail-Adresse :	lennart.mende@stud.htwk-leipzig.de

und

vorgelegt von:	Richard Mende
Studiengang:	Elektrotechnik und Informationstechnik
Matrikelnummer:	81169
E-Mail-Adresse :	richard.mende@stud.htwk-leipzig.de

Datum der Abgabe:	31. Oktober 2026

Inhaltsverzeichnis

Abbildungsverzeichnis **II**

Tabellenverzeichnis **II**

1 Einleitung **1**
1.1 Zielstellung . . . 1
1.2 Hinweise . . . 1

2 Praktikumsversuch mit dem TurtleBot 4 **2**
2.1 Einrichtung der Arbeitsumgebung . . . 2
2.2 Manuelle Steuerung . . . 2
2.3 Erweiterte Steuerungsmethode . . . 2
2.4 Kartierung und autonome Navigation . . . 2
2.5 Verhalten des Roboters während der Navigation . . . 4

3 Steuerung und Programmierung des Roboters **5**
3.1 Bewegungssteuerung für die Turtlesim und den Roboter . . . 5
3.1.1 Steuerung der Turtlesim durch parametrierbare Bewegungsbefehle . . . 5
3.1.2 Übertragung der Bewegungssteuerung auf den realen Roboter . . . 7
3.2 Entwicklung des Projekts für den Roboter . . . 10
3.2.1 Einbindung der Sensorik und Aktorik . . . 10
3.2.2 Kommunikation zwischen Host-PC und ESP32 . . . 11
3.2.3 Beschreibung des implementierten Algorithmus . . . 12
3.2.4 Demonstration . . . 15

4 Programmieraufgaben Echtzeitsysteme **16**
4.1 Sockets und Netzwerkkommunikation . . . 16
4.1.1 Server . . . 16
4.1.2 Client . . . 18
4.1.3 Verwaltung einer gemeinsamen Liste . . . 19
4.1.4 Peer-to-Peer-Erweiterung . . . 23
4.2 IPC . . . 25
4.2.1 Server-Prozess . . . 25
4.2.2 Client-Prozess . . . 25
4.2.3 Verwaltung einer gemeinsamen Liste . . . 26
4.2.4 Gleichberechtigte Kommunikation . . . 27

5 Zusatzaufgaben **29**
5.1 Zusatzaufgabe 1 . . . 29
5.2 Zusatzaufgabe 2 . . . 30

6 Herausforderungen und mögliche Verbesserungen **32**

Literaturverzeichnis **III**

Abbildungsverzeichnis

2.1 Aufgezeichnete Karte in RViz 3
2.2 Autonome Navigation mit Start- und Zielpunkt in RViz 3
3.1 Kreisförmige Bewegung der Turtle 6
3.2 Viereckige Bewegung der Turtle 7
3.3 Navigation in VSC für das PIO-Projekt 9
3.4 Ultraschallsensor mit Spannungsteiler 10

Tabellenverzeichnis

3.1 Topics der Sensorwerte 11
3.2 Topics der Aktorwerte 11
3.3 Übersicht der Methoden für die Sensorabfragen 13
3.4 Übersicht der Methoden für die Ansteuerung der Aktoren 13
4.1 Übersicht der Aktionen beim Trennen einzelner Teilnehmer 24

1 Einleitung

Das einleitende Kapitel dient zur Beschreibung der Zielstellung dieses Belegs. Außerdem werden Hinweise zur Struktur der eingereichten ZIP-Datei gegeben.

1.1 Zielstellung

Innerhalb des Moduls *Echtzeitsysteme und mobile Robotik* sind die drei folgenden Aufgaben für den Beleg zu bearbeiten:

1. Durchführung des Praktikumsversuchs mit dem TurtleBot 4
2. Programmierung eines bereitgestellten Roboters mit einem ausgewählten Projekt
3. Programmieraufgaben zu Echtzeitsystemen

Dieser Beleg dient zur Dokumentation der durchgeführten Aufgaben in den folgenden Kapiteln.

1.2 Hinweise

Der Beleg wurde als ZIP-Datei `EZMR_Mende_Mende` eingereicht. Beim Entpacken dieser Datei entsteht eine Ordnerstruktur mit `EZMR_Mende_Mende` als Root-Verzeichnis. Darin befinden sich die folgenden Unterordner mit den jeweiligen Inhalten:

- `00_documentation`: enthält diese PDF-Datei zur Dokumentation der durchgeführten Aufgaben
- `01_turtlebot_lab`: enthält die Videos, die während des Praktikumsversuchs mit dem TurtleBot 4 aufgenommen wurden
- `02_turtlesim_task`: enthält das ROS 2 Paket für die Steuerung der Turtle
- `03_roboter_project`: enthält den Quellcode für das Projekt, das für den ausgegebenen Roboter implementiert wurde
- `04_programming_exercises`: enthält den Quellcode für die Programmieraufgaben
- `05_additional_tasks`: enthält die Dateien für die Umsezung der Zusatzaufgaben

In diesem Beleg wird mehrfach auf den Abgabeordner verwiesen. Dabei wird davon ausgegangen, dass man sich bereits im Root-Verzeichnis `EZMR_Mende_Mende` befindet. Somit werden bei diesen Pfadangaben die Unterordner `00_documentation`, `01_turtlebot_lab`, `02_turtlesim_task`, `03_roboter_project`, `04_programming_exercises` sowie `05_additional_tasks` als Startpunkt für die Pfadangaben verwendet.

Darüber hinaus wurden sowohl für das Roboter-Projekt als auch für die Programmieraufgaben GitHub-Repositories erstellt, um die Versionskontrolle sowie eine nachvollziehbare Zusammenarbeit zu gewährleisten. Der Inhalt der Ordner `03_roboter_project` und `04_programming_exercises` sind identisch dem Inhalt der GitHub-Repositories unter den Links `https://github.com/LennartMende/RealtimeSystemsMobileRobotics` bzw. `https://github.com/rimen13/programming-exercises-realtime-systems`.

Repos (ausmisten und) von private auf public umstellen, alternativ

2 Praktikumsversuch mit dem TurtleBot 4

In diesem Kapitel wird die Durchführung des Praktikumsversuchs mit dem professionellen Roboter TurtleBot 4 beschrieben. Auf die einzelnen Schritte der Praktikumsdurchführung wird in chronologischer Reihenfolge in den folgenden Abschnitten genauer eingegangen.

2.1 Einrichtung der Arbeitsumgebung

Um den TurtleBot einsatzbereit zu machen, wurden die in [1] angegebenen Anweisungen aus Kapitel 3 umgesetzt. Anschließend erfolgte die Sicherstellung der Funktionsfähigkeit des TurtleBot mit den drei Befehlen `ros2 node list`, `ros2 topic list` und `ros2 topic echo /battery_state`. Durch die ersten beiden Befehle werden die im ROS 2 Netzwerk sichtbaren Nodes (z. B. `/analyzers`) und Topics (z. B. `/battery_state`) aufgelistet. Der letzte Befehl wurde sowohl zu Beginn als auch während des Praktikums regelmäßig genutzt, um den Ladezustand des Akkus zu überprüfen.

2.2 Manuelle Steuerung

Nach dem Hochfahren des TurtleBot läuft auf dessen Raspberry Pi die für die Ansteuerung des Roboters benötigte ROS 2 Software. Diese abonniert unter anderem das Topic `/cmd_vel`, über das Geschwindigkeitsbefehle an den Roboter übertragen werden. Durch Starten der Teleoperation mit dem Befehl `ros2 run teleop_twist_keyboard teleop_twist_keyboard` kann der Bediener mit der Tastatur die Steuerungsbefehle für den Roboter vorgeben. Die Steuerungsbefehle werden über ROS 2 im Netzwerk veröffentlicht. Mit `ros2 topic echo /cmd_vel` können die veröffentlichten Werte für die lineare Geschwindigkeit und Winkelgeschwindigkeit angezeigt werden.

2.3 Erweiterte Steuerungsmethode

Um die Anforderungen aus Abschnitt 2.5 der Praktikumsanleitung [2] zu erfüllen, wurde ein ROS 2-Paket erstellt. Eine ausführliche Beschreibung dieses Pakets erfolgt in Abschnitt 3.1. Beim Aufruf des Befehls `ros2 run turtlesim_task turtlesim_publisher` erfolgt das Abfahren eines N-Ecks durch den TurtleBot. Dabei kann N durch die Erweiterung des genannten Befehls mit `-ros-args -p num_sides:=4` beispielsweise auf 4 gesetzt werden. Für den TurtleBot wurden das Dreieck, Viereck und auch Zehneck erfolgreich getestet. Im Abgabeordner ist unter `01_turtlebot_lab/drive_square.mp4` ein Video abgelegt, welches das korrekte Abfahren des Vierecks durch den TurtleBot demonstriert.

2.4 Kartierung und autonome Navigation

Abschließend erfolgte eine Kartierung der Umgebung mithilfe des TurtleBot und die anschließende autonome Navigation auf dieser erstellten Karte. Entsprechend der Anweisung aus Abschnitt 6.1 von [1] wurde dafür zunächst SLAM (Simultaneous Localization and Mapping) und anschließend das Visualisierungstool RViz2 mit der in 6.2 in [1] beschriebenen Konfiguration gestartet. Zur Erstellung der Karte wurden schließlich die Schritte aus [3] befolgt. Abbildung 2.1 zeigt das RViz-Fenster nach dem Aufzeichnen der Karte. Mithilfe der in [3] beschriebenen Befehle erfolgte das Speichern der Karte als PGM- sowie YAML-Datei. Diese Dateien dienten als Grundlage für die Navigation. Um den Roboter autonom vom Start zum Ziel zu navigieren, wurden nach [4] die Start- und Zielposition im Navigationsfenster von RViz vorgegeben, wie in Abbildung 2.2 zu sehen ist.

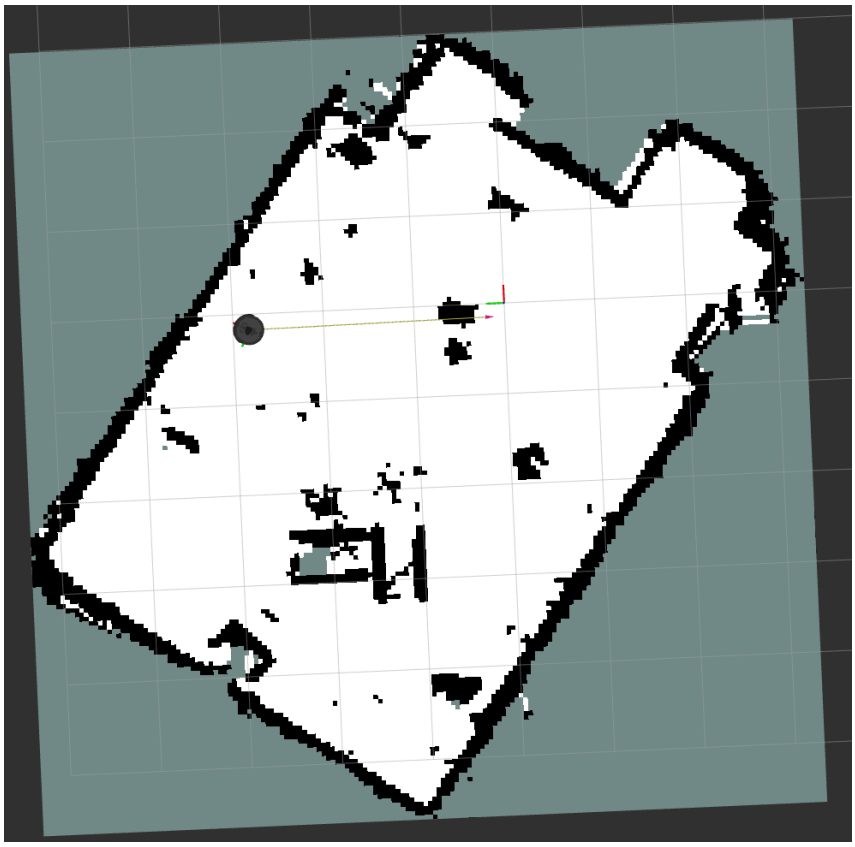


Abbildung 2.1: Aufgezeichnete Karte in RViz



Abbildung 2.2: Autonome Navigation mit Start- und Zielpunkt in RViz

2.5 Verhalten des Roboters während der Navigation

Im Abgabeordner befindet sich unter `01_turtlebot_lab/nav_to_goal.mp4` ein Video, in welchem die autonome Navigation des TurtleBot beispielhaft demonstriert wird. Dabei ist zu erkennen, dass der TurtleBot die bereits kartierte Umgebung berücksichtigt und bekannten Hindernissen ausweicht. Zudem passt er seine Geschwindigkeit an die Umgebung an, indem er beispielsweise bei einer Annäherung an Hindernisse langsamer fährt als auf geraden Strecken.

Allerdings zeigt sich eine Einschränkung bei der Reaktion auf dynamische Veränderungen der Umgebung, wie beispielhaft im Video unter `01_turtlebot_lab/nav_obstacle.mp4` zu sehen ist. Dabei stellt sich eine Person plötzlich direkt vor den TurtleBot. Dieser erkennt das Hindernis in diesem Fall nicht rechtzeitig und fährt gegen den Fuß der Person. Eine mögliche Ursache hierfür ist eine zu geringe Aktualisierungs- oder Abtastrate der relevanten Sensordaten beziehungsweise eine zu lange Verarbeitungs- und Reaktionszeit. Möglicherweise könnte auch die Programmierung des Roboters ein weiterer Grund für das gezeigte Verhalten sein.

Neben den bereits während der Kartierung erfassten Hindernissen kann der TurtleBot auch nachträglich hinzugefügte statische Hindernisse erkennen. Werden beispielsweise Gegenstände nach der Kartierung in den Fahrweg gestellt, kann der TurtleBot diese während der Navigation erfassen und ihnen ausweichen.

3 Steuerung und Programmierung des Roboters

Dieses Kapitel beinhaltet die Umsetzung einer parametrierbaren Bewegungssteuerung der Turtlesim aus [5] und die Übertragung auf den in Abschnitt 4.2 der Praktikumsanleitung [2] beschriebenen Roboter. Darauf aufbauend erfolgte die Entwicklung eines eigenständigen Projekts zur Navigation durch ein Labyrinth mithilfe der Sensorik und Aktorik des Roboters. Zur Kommunikation wurde ROS 2 Humble verwendet.

3.1 Bewegungssteuerung für die Turtlesim und den Roboter

3.1.1 Steuerung der Turtlesim durch parametrierbare Bewegungsbefehle

Um die in Abschnitt 2.3 der Praktikumsanleitung [2] gegebenen Aufgabe zu erfüllen, wurde ein ROS 2 Paket zur Steuerung der Turtlesim erstellt. Dieses Paket mit dem Namen `turtlesim_task` befindet sich im Abgabeordner unter `02_turtlesim_task`. Die implementierte Bewegungssteuerung ist in der Quelldatei `src/turtlesim_publisher.cpp` implementiert. Dabei werden Nachrichten des Typs `geometry_msgs/msg/Twist` als Steuerungsbefehle über das Topic `/cmd_vel` veröffentlicht. Die Steuerungsbefehle sind dabei die lineare Geschwindigkeit und angulare Geschwindigkeit für die Turtle. Es werden jeweils die x-, y- und z-Komponente für Translation und Rotation veröffentlicht. Allerdings sind nur die x-Komponente der linearen Bewegung und die z-Komponente der Rotation relevant, da alle anderen Komponenten stets 0 sind. Durch die Wahl dieser Geschwindigkeitswerte kann die Bewegungsart vorgegeben werden, beispielsweise eine geradlinige Bewegung, Dreh- oder Kreisbewegung.

Die Implementierung ermöglicht das Abfahren eines Kreises oder eines N-Ecks. Für $N < 3$ führt die Turtle eine Kreisbewegung aus. In diesem Fall werden die vorgegebene lineare Geschwindigkeit und Winkelgeschwindigkeit fortlaufend gleichzeitig veröffentlicht. Für $N \geq 3$ fährt die Turtle die Seiten des entsprechenden N-Ecks ab. Dafür wird eine Zustandsmaschine verwendet, die zwischen einer geradlinigen Bewegung und einer Drehbewegung wechselt. Zunächst fährt die Turtle für eine vorgegebene Dauer mit der vorgegebenen linearen Geschwindigkeit geradeaus. Anschließend dreht sie sich um den für die gewünschte Form erforderlichen Winkel von $2\pi/N$. Die Dauer der Drehung wird aus dem Quotienten des Drehwinkels und der vorgegebenen Winkelgeschwindigkeit berechnet.

Der Anwender kann beim Aufruf der Node die Werte für 4 verschiedene Parameter vorgeben und somit die Node konfigurieren. Dazu zählen

1. die Anzahl der Seiten des N-Ecks `num_sides`
2. die Dauer der geradlinigen Bewegung `forward_duration`
3. die lineare Geschwindigkeit `linear_vel`
4. die Winkelgeschwindigkeit `angular_vel`

Die konkreten Parameterwerte können beim Aufruf der Node vorgegeben werden, z. B.:

```
ros2 run turtlesim_task turtlesim_publisher \
  --ros-args \
  -p num_sides:=4 \
  -p forward_duration:=5.0 \
  -p linear_vel:=0.5 \
  -p angular_factor:=1.0
```

Damit wird ein Viereck mit einer Vorwärtsfahrtdauer von 5 s, einer linearen Geschwindigkeit von 0,5 m/s und einer Winkelgeschwindigkeit von 1 π/s vorgegeben. Der letzte Parameter `angular_factor` gibt dementsprechend den Vorfaktor für die Winkelgeschwindigkeit an und wird noch mit π multipliziert. Ohne zusätzliche Angabe beim Aufruf gelten die Standardwerte $N = 2$, eine Vorwärtsfahrtdauer von 3 s, eine lineare Geschwindigkeit von 1 m/s und ein Winkelgeschwindigkeitsfaktor von 0,25.

Um dieses Paket und die implementierte Steuerungsmethode für die Turtlesim verwenden zu können, wird eine installierte und gesourcte ROS 2 Umgebung in jedem neuen Terminal nach [6] vorausgesetzt. Außerdem muss das beschriebene ROS 2 Paket `turtlsim_task` in den `src` Ordner dieses Workspaces installiert werden. Es werden 2 Terminals gestartet und jeweils in den ROS 2 Workspace gewechselt. Im ersten Terminal werden mit

```
colcon build --packages-select turtlesim_task
ros2 run turtlesim turtlesim_node
```

das ROS 2 Paket gebaut und die Turtlesim gestartet und im zweiten Terminal die Bewegungssteuerung mit

```
source install/setup.bash
ros2 run turtlesim_task turtlesim_publisher
```

vorgegeben. Um die konkreten Parameterwerte zu übergeben, kann der oben genannte Befehl verwendet werden. Anschließend sieht man im zweiten Terminal jede Sekunde die veröffentlichten Geschwindigkeitswerte. Außerdem sieht man im durch das erste Terminal geöffnete „TurtleSim“ Fenster die Bewegung der Turtle. In Abbildung 3.1 ist die aus dem hier angegebenen Bewegungsbefehl resultierende Bewegung der Turtle dargestellt. Hier ist der von der Turtle gezeichnete Kreis für jeden Durchlauf nahezu konstant.

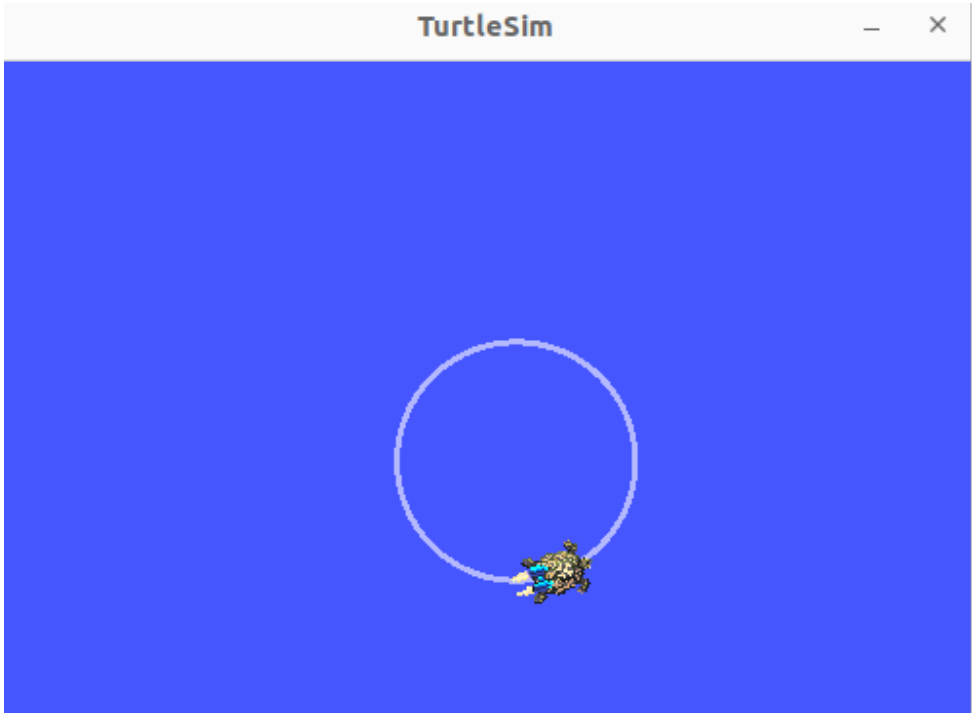


Abbildung 3.1: Kreisförmige Bewegung der Turtle

Abbildung 3.2 zeigt die Bewegung der Turtle für den viereckigen Bewegungsablauf mit dem oben genannten Befehl für die angepassten Parameterwerte. Hierbei ist zu erkennen, dass sich das Quadrat in jedem Durchlauf etwas verschiebt. Dies liegt an den Ungenauigkeiten der in der Zustandsmaschine verwendeten Timer, weswegen das Timing für die Dauer der geradlinigen Bewegung und der Drehbewegung in der Zustandsmaschine nicht exakt ist. Außerdem können Verzögerungen in der ROS 2 Kommunikation zu Abweichungen führen.

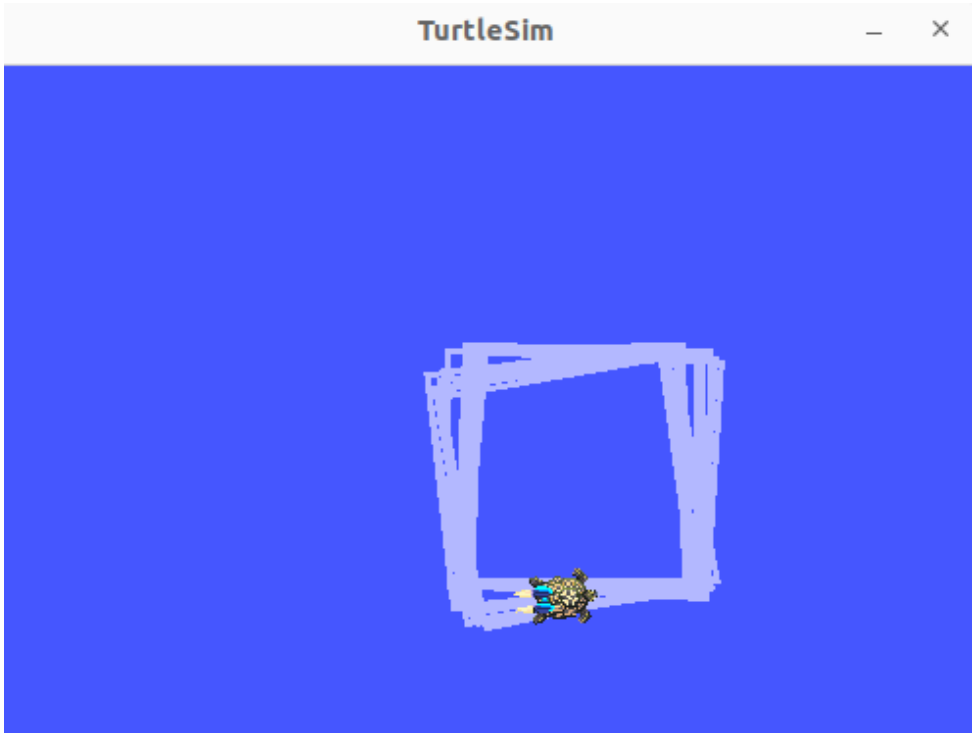


Abbildung 3.2: Viereckige Bewegung der Turtle

3.1.2 Übertragung der Bewegungssteuerung auf den realen Roboter

Eine weitere Aufgabe bestand darin, die Funktionsweise des in Abschnitt 3.1.1 beschriebenen ROS 2 Pakets für die Bewegungssteuerung der Turtle auf den eigenen Roboter zu übertragen. Dafür wurde die neue Quelldatei unter `03_roboter_project/ros2_ws/src/actuators/src/cmd_vel_publisher.cpp` erstellt. Die grundsätzlichen Funktionalitäten bleiben erhalten, nur dass hier die lineare Geschwindigkeit und die Winkelgeschwindigkeit für den Schrittmodus fest vorgegeben sind. Die kontinuierliche Steuerungsmethode hat nicht reproduzierbar funktioniert, wie in Abschnitt 6 genauer erläutert wird. Die Messwerte für die Geschwindigkeiten wurden mithilfe von wiederholten Messungen ermittelt. Dabei kommt es Ungenauigkeiten für die Messung der Distanz und Zeit. Außerdem ist festzuhalten, dass der Roboter nicht exakt gerade fährt und die Räder gelegentlich an dem Rahmen reiben. Darüber hinaus starten die beiden Räder zu unterschiedlichen Zeitpunkten. Das linke Rad startet stets zeitiger als das rechte Rad. Als Ergebnis ergaben sich bei 1000 Steps im Durchschnitt eine Entfernung von 12,5 cm und eine Zeit von 3,0 s und damit eine lineare Geschwindigkeit von $v_x = 4{,}16 cm/s$ bei $\omega_z = 0$. Für die Winkelgeschwindigkeit ergab sich im Durchschnitt $\omega_z = 0{,}15\pi/s$ bei $v_x = 0$. Diese Werte wurden in der genannten Quelldatei als Konstanten definiert, weshalb diese Parameter beim Aufruf in der Kommandozeile nicht übergeben werden können.

Der Ablauf zum Ausführen der gewünschten Bewegungen ist ähnlich zu dem in Abschnitt 3.1.1 beschriebenen Ablauf. Dafür wird zunächst in den ROS 2 Workspace dieses Projekts unter `03_roboter_project/ros2_ws` gewechselt. Anschließend erfolgt das Bauen des ROS 2 Pakets `actuators` mit

```
colcon build --packages-select actuators
```

In einem neuen Terminal erfolgt das Sourcen von Micro-ROS im angelegten Workspace und das Starten des Micro-ROS-Agenten mit den Befehlen

```
source micro_ros_ws/install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8888
```

Außerdem muss eine zusätzliche Datei angelegt werden, die nicht im Abgabeordner enthalten ist. Dabei handelt es sich um die Datei `03_roboter_project/include/secrets.hpp`. Diese wurde aus Sicherheitsgründen nicht abgegeben, da sie die SSID des WLANs und das WLAN-Passwort sowie die IP-Adresse des Host-PCs enthält. Diese Datei muss im angegebenen Pfad mit den persönlichen Daten mit folgendem Inhalt ergänzt werden:

```
#pragma once
constexpr char WIFI_SSID[] = "<WLAN_NAME>";
constexpr char WIFI_PASSWORD[] = "<WLAN_PASSWORT>";
constexpr uint8_t AGENT_IP[4] = {<IP_ADRESSE>};
constexpr int AGENT_PORT = 8888;
```

Die `<IP_ADRESSE>` muss beispielweise im Format `192, 168, 255, 255` angegeben werden.

Anschließend erfolgt das Flashen der Firmware auf den ESP. Als Ausgangspunkt wird dafür vorausgesetzt, dass die notwendigen Einrichtungsschritte für PlatformIO in Visual Studio Code (VSC) aus Abschnitt 4.1.4 der Praktikumsanleitung [2] bereits vorgenommen wurden. Außerdem müssen der ESP und der Host-PC über USB verbunden sein und die letzten beiden Zeilen

```
upload_protocol = espota
upload_port = 192.168.255.255
```

in der Datei `03_roboter_project/platformio.ini` auskommentiert werden. Falls bereits eine erfolgreiche OTA-Verbindung eingerichtet ist, müssen diese Zeilen weder auskommentiert noch die Verbindung über USB hergestellt werden. Es muss aber die korrekte IP-Adresse des ESP für den `upload_port` eingetragen werden. In der Seitenleiste in „PlatformIO“ unter „Project Tasks - esp32dev - General“ die Möglichkeit, das Projekt mit „Build“ zu kompilieren und die Firmware mit „Upload“ auf den ESP zu übertragen. Diese Navigation zeigt Abbildung 3.3.

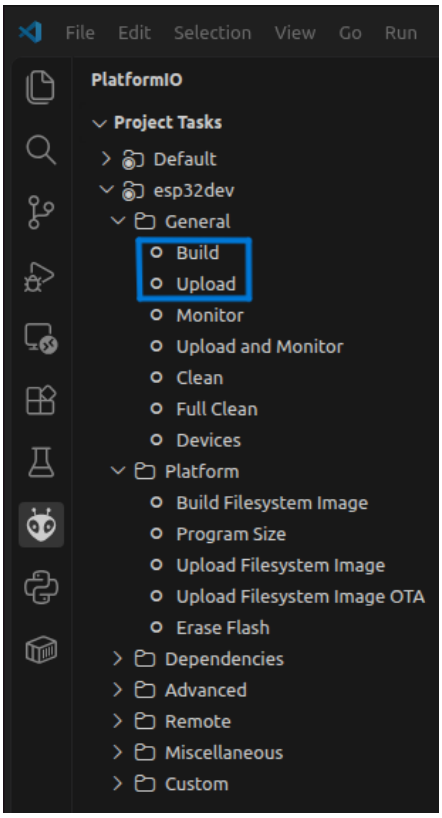


Abbildung 3.3: Navigation in VSC für das PIO-Projekt

Das Starten des Publishers für die Bewegungsbefehle des Roboters erfolgt in einem Terminal im Ordner des ROS 2 Workspaces mit den Befehlen

```
source install/setup.bash
ros2 run actuators cmd_vel_publisher \
  --ros-args \
  -p num_sides:=4 \
  -p forward_duration:=5.0
```

In diesem Fall wird ein Viereck mit einer Vorwärtsdauer von 5 s abgefahren. Beim Ausführen der Bewegungen ist zu beachten, dass die Bewegung des Roboters nicht kontinuierlich erfolgt. Dies liegt daran, dass die Geschwindigkeitsbefehle alle 100 Millisekunden veröffentlicht werden. Es wurde sich für dieses Intervall entschieden, da mit einer geringeren Dauer ein schlechteres Ergebnis erzielen würde. Bei einem größeren Publish-Intervall wäre hingegen die Reaktionsheit zu groß, sodass beispielsweise Drehungen zu lang andauern würden. Aus dem gegebenen Publish-Intervall werden die Steps für die Motoren berechnet und an den ATtiny weitergegeben. Als Resultat ist zu erkennen, dass der Roboter kein exaktes Viereck abfährt. Dies liegt vor allem am nicht synchronen Starten der Bewegungen des linken und rechten Rads. Durch das verhältnismäßig große Publish-Intervall können die Geschwindigkeitsbefehle dazu führen, dass die Drehung nicht 90 Grad beträgt. Dementsprechend ergibt sich eine ähnliche Bewegungsabfolge wie bei der Turtlesim in Abbildung 3.2. Für andere Formen wie Kreis, Dreieck, Fünfeck usw. wurde die Bewegungsabfolge ebenfalls getestet. Auch hier zeigen sich die Auswirkungen der genannten Hardware-Probleme. An dieser Stelle hätte mit einem kontinierlichen Modus statt einem Step-Modus für die Motoren ein deutlich besseres Ergebnis erzielt werden können.

3.2 Entwicklung des Projekts für den Roboter

Eine weitere Projektaufgabe bestand in der Programmierung des zur Verfügung gestellten Roboters mit einer vorzugebenden Aufgabe. Dafür wurde die Navigation durch ein Labyrinth mithilfe des angepassten Pledge-Algorithmus ausgewählt. Dieser beschreibt den folgenden Ablauf zum Durchfahren des Labyrinths. Zunächst fährt der Roboter von seinem Startpunkt geradeaus, bis er sich direkt vor einer Wand befindet. Anschließend dreht er sich um 90 Grad im Uhrzeigersinn und folgt dieser Wand, die sich nun links von ihm befindet. Das Erreichen des Ziels muss von außen vorgegeben werden, da es nicht anhand der Sensordaten erkannt werden kann. Die Umsetzung dafür wird in Abschnitt 3.2.4 beschrieben. Die Voraussetzung für die korrekte Funktionsweise des Pledge-Algorithmus ist, dass Start und Ziel über eine Außenwand verbunden sind. [7]

3.2.1 Einbindung der Sensorik und Aktorik

Für die Sensorik wurden der bereits vorhandene Ultraschallsensor für die Abstandsmessung zu Hindernissen vor dem Roboter sowie die beiden Bumper für die Erkennung von Kollisionen des Roboters mit Hindernissen genutzt. Zusätzlich wurde ein zweiter Ultraschallsensor am linken Rad befestigt, um den Abstand zu Hindernissen auf der linken Seite messen zu können und die Umsetzung des Projekts zu ermöglichen. Da der ESP nur 3.3 V-Signale verarbeiten kann, aber der Ultraschallsensor ein mit 5 V skaliertes Signal als Echo liefert, wurde dieses mithilfe eines Spannungsteilers auf 3.3 V umgewandelt. In Abbildung 3.4 ist der zusätzlich angebrachte Ultraschallsensor mit dem Spannungsteiler auf dem Breadboard dargestellt. Für den zweiten Ultraschallsensor wird Pin 23 als Echo und Pin 27 als Trigger verwendet, da diese beiden Pins noch verfügbar sind und die notwendigen Anforderungen erfüllen.

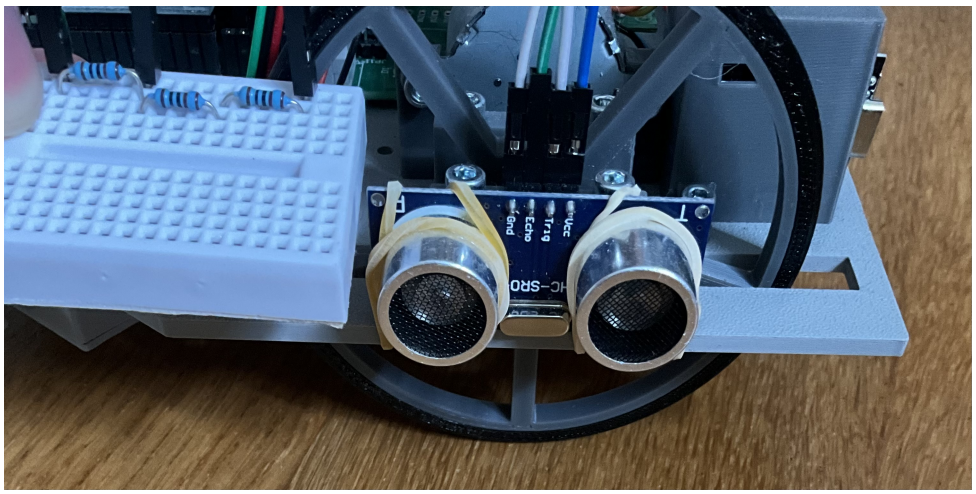


Abbildung 3.4: Ultraschallsensor mit Spannungsteiler

Für die Umsetzung des Projekts wurden ausschließlich Topics für die Kommunikation verwendet. Tabelle 3.1 stellt eine Übersicht über die Topics dar, auf denen die Sensorwerte gestreamt werden. Weiterhin sind die zugehörigen Interfaces und die Bedeutung der Topics aufgelistet. Für diese Topics der Sensorwerte wurden Publisher in Micro-ROS sowie Subscriber in ROS 2 implementiert.

Tabelle 3.1: Topics der Sensorwerte

Topic	Interface	Beschreibung
/bumpers	robot_interfaces/msg/BumpersStates	Bool-Werte für die Bumper (true = gedrückt)
/sonar_front	std_msgs/msg/Int16	gemessener Abstand des vorderen Ultraschallsensors in cm
/sonar_left	std_msgs/msg/Int16	gemessener Abstand des linken Ultraschallsensors in cm

Für die Aktorik wurden die Schrittmotoren zur Steuerung der Bewegung des Roboters genutzt. Während einer Drehung blinkt die LED und der Buzzer piept. Die OLED dient zur Anzeige der aktuellen Steuerungsmethode. Tabelle 3.2 stellt eine Übersicht über die Topics dar, auf denen die Aktorwerte gestreamt werden. Außerdem sind die zugehörigen Interfaces und die Bedeutung der Topics aufgelistet. Für diese Topics der Aktorwerte wurden Publisher in ROS 2 sowie Subscriber in Micro-ROS implementiert.

Tabelle 3.2: Topics der Aktorwerte

Topic	Interface	Beschreibung
/motors_cmd	robot_interfaces/msg/MotorsCmd	Werte für Drehrichtungen und Geschwindigkeiten der Motoren
/led_state	std_msgs/msg/Bool	Zustand der LED (true = an)
/buzzer_state	std_msgs/msg/Bool	Zustand des Buzzers (true = an)
/oled_text	std_msgs/msg/String	auf der OLED angezeigter Text

3.2.2 Kommunikation zwischen Host-PC und ESP32

Für die Entwicklung des Projekts werden grundlegend die Firmware des ESP32 und die ROS 2 Anwendung auf dem Host-PC unterschieden. Der ESP32 ist für den Hardware-Zugriff auf die Sensoren und Aktoren verantwortlich, während die ROS 2 Anwendung auf dem Host-PC die übertragenen Sensorwerte verarbeitet und daraus die Befehle für die Aktoren bestimmt. Die Kommunikation zwischen den beiden Komponenten erfolgt über Micro-ROS. Der ESP32 veröffentlicht die Sensorwerte über Publisher und empfängt über Subscriber die Befehle für die Aktoren. Auf dem Host-PC werden die Sensorwerte über Subscriber entgegengenommen und die Aktorbefehle über Publisher übertragen. Dadurch sind die Hardwareansteuerung und die Navigationslogik voneinander getrennt.

Im Abgabeordner ist unter `03_roboter_project` die folgende Ordnerstruktur mit dem zugehörigen Inhalt angelegt:

- `include`: beinhaltet die Header-Dateien für die Quelldateien im `src`-Ordner
- `src`: beinhaltet den Quellcode für die Firmware des ESP zur Ansteuerung der Aktorik und Abfrage der Sensorik sowie für die Kommunikation überMicro-ROS
- `ros2_ws`: ROS 2 Workspace, enthält den Quellcode und die Header-Dateien für das Ausführen auf dem Host-PC

Der Hardwarezugriff auf die Sensorik und Aktorik über den ESP wurde mithilfe des Arduino-Frameworks programmiert. Zum Debuggen über die serielle Schnittstelle kann außerdem die Variable `USB_DEBUG` ider Quelldatei `include/microros/error_handling.hpp` gesetzt werden. Dafür müssen der ESP und der Host-PC über USB verbunden sein und die Schritte aus Abschnitt 3.1.2 zum Flashen des ESP ausgeführt werden.

Für den ESP existieren separate Klassen für Aktorik und Sensorik. Diese Klassen werden in der `MicroRos`-Klasse unter `03_roboter_project/src/microros/microros.cpp` instanziiert. Mithilfe dieser Objekte kann man den Hardwarezugriff und die Micro-ROS-Kommunikation realisieren. Ein Executor ermöglicht außerdem, dass die Publisher und Subscriber der Node dauerhaft gespinnt werden. Somit kann jederzeit auf Timer und ROS-basierte Events reagiert und Callbacks ausgeführt werden.

Auf dem Host-PC bündelt die Klasse `ControllerNode` die ROS 2 Publisher und Subscriber für die Sensor- und Aktordaten. Dadurch stellt sie eine einheitliche Schnittstelle bereit, über die der Pledge-Algorithmus Sensordaten abfragen und Steuerungsbefehle veröffentlichen kann. Die implementierte Logik für den Pledge-Algorithmus auf dem Host-PC wird im nächsten Abschnitt beschrieben.

3.2.3 Beschreibung des implementierten Algorithmus

Die `ControllerNode` ist in `03_roboter_project/ros2_ws/src/algorithm/src/robot_controller.cpp` implementiert. Für diese Klasse sind weitere Methoden definiert, welche das Abfragen von Sensorwerten sowie das Senden von Befehlen für die Aktoren ermöglichen. Diese sind in den Tabellen 3.3 und 3.4 aufgeführt. Die als *public* deklarierten Methoden können in der Quelldatei `03_roboter_project/ros2_ws/src/algorithm/src/pledge_algorithm.cpp` aufgerufen werden, um die Ablauflogik des Pledge-Algorithmus umzusetzen. Die Implementierung des Pledge-Algotihmus in der genannten Quelldatei wird im Folgenden ausführlich beschrieben. Die `ControllerNode` wird in `03_roboter_project/ros2_ws/src/algorithm/src/robot_controller_main.cpp` initialisiert sowie gespinnt.

Tabelle 3.3: Übersicht der Methoden für die Sensorabfragen

Methode	**Bedeutung**
`getSonarFrontValue()`	gibt den aktuellen Messwert des vorderen Ultraschallsensors in cm zurück
`getSonarLeftValue()`	gibt den aktuellen Messwert des linken Ultraschallsensors in cm zurück
`getLastWallSample()`	gibt den letzten gespeicherten Messwert der linken Wandmessung in cm zurück
`isLeftPressed()`	prüft, ob der linke Bumper aktuell gedrückt ist
`isRightPressed()`	prüft, ob der rechte Bumper aktuell gedrückt ist
`wasLeftPressed()`	prüft, ob der linke Bumper seit der letzten Auswertung gedrückt wurde
`wasRightPressed()`	prüft, ob der rechte Bumper seit der letzten Auswertung gedrückt wurde
`getImuValues()`	gibt die verfügbaren Messwerte vom IMU zurück

Tabelle 3.4: Übersicht der Methoden für die Ansteuerung der Aktoren

Methode	**Bedeutung**
`setOled(text)`	gibt den übergebenen Text auf dem OLED-Display aus
`moveForwardExact (distance_cm)`	bewegt den Roboter über eine angegebene Strecke vorwärts
`moveBeforeLeftTurn (last_wall_sample)`	berechnet anhand des übergebenen Werts eine zusätzliche Vorwärtsstrecke vor einer Linksdrehung
`moveForward()`	bewegt den Roboter für eine festgelegte Zeit vorwärts
`moveBackward()`	bewegt den Roboter für eine festgelegte Zeit rückwärts
`turnClockwise()`	dreht den Roboter auf der Stelle im Uhrzeigersinn und signalisiert die Drehung durch LED und Summer
`turnCounterClockwise()`	dreht den Roboter auf der Stelle gegen den Uhrzeigersinn und signalisiert die Drehung durch LED und Summer
`setBuzzer(on)`	schaltet den Summer ein oder aus
`setLed(on)`	schaltet die LED ein oder aus
`setMotorsCmd(dir_left, dir_right, steps)`	übermittelt den Drehrichtungen der beiden Motoren und der Schrittanzahl entsprechende Bewegungsbefehle

Der Algorithmus beginnt mit dem Initialschritt. Dabei fährt der Roboter in 20 cm Schritten geradeaus. Nach jedem Absolvieren dieser 20 cm wird überprüft, ob der Abstand nach vorn (Messwert vom vorderen Ultraschallsensor) kleiner als 30 cm ist. Sobald dieser kleiner ist als 30 cm, fährt der Roboter bis auf ca. 10 cm an die Wand. Dabei entstehen geringe Abweichungen einerseits durch Sensorungenauigkeiten und die Auflösung sowie durch die in Abschnitt 3.1.2 beschriebenen Hardware-Ungenauigkeiten des Roboters. Die Zeit für diesen Schritt wird mithilfe der bekannten linearen Geschwindigkeit aus Abschnitt 3.1.2 berechnet. Wenn diese Zeit abgelaufen ist, dreht sich der Roboter im Uhrzeigersinn, sodass sich die Wand schließlich auf seiner linken Seite befindet.

Dieser Wand folgt der Roboter anschließend, indem er sich um 10 cm Schritte vorwärts bewegt. Anhand der Sensorwerte des linken Ultraschallsensor wird dabei festgestellt, ob

die Wand auf der linken Seite weiterhin vorhanden ist. Wenn der linke Ultraschallsensor bei zwei aufeinanderfolgenden Abtastungen hintereinander mehr als 30 cm misst, wird angenommen, dass sich links neben dem Roboter keine Wand mehr befindet. Hierbei werden zwei aufeinanderfolgende Abtastungen betrachtet, um den Einfluss fehlerhafter Sensorwerte zu veringern (beispielsweise wird gelegentlich ein fehlerhafter Messwert von 0 gesendet). Gleichzeitig wird mit den Werten des vorderen Ultraschallsensors überprüft, ob sich vorne eine Wand befindet.

Die Steuerung des Roboters wurde als endliche Zustandsmaschine (FSM) programmiert. Dabei wird nach jedem der 10 cm Schritte der aktuelle Zustand auf Basis der während des letzten Schritts aufgenommenen Sensordaten überprüft. Es gibt vier grundlegende Zustände, die sich aus den Messwerten der beiden Ultraschallsensoren ergeben können:

(a) links befindet sich eine Wand, vorne befindet sich keine Wand

(b) links befindet sich keine Wand, vorne befindet sich keine Wand

(c) links befindet sich eine Wand, vorne befindet sich eine Wand

(d) links befindet sich keine Wand, vorne befindet sich eine Wand

Die FSM führt für jeden der vier Zustände die jeweils definierten Aktionen aus. Da der Roboter nicht erkennen kann, ob er sein Ziel erreicht hat, läuft die Zustandslogik kontinuierlich weiter. Das Programm wird nur dann beendet, wenn der Roboter gegen ein Hindernis fährt (das zu klein ist, um vom vorderen Ultraschallsensor erkannt zu werden) und dies durch die Bumper registriert wird. Die Aktionen für die 4 Zustände werden in der folgenden Fallunterscheidung erläutert.

(a) links befindet sich eine Wand, vorne befindet sich keine Wand

Diese Bedingung ist erfüllt, wenn der Abstand nach vorne größer als 20 cm und nach links kleiner gleich 20 cm ist. In diesem Fall fährt der Roboter 10 cm vorwärts.

(b) links befindet sich keine Wand, vorne befindet sich keine Wand

Diese Bedingung ist erfüllt, wenn der Abstand nach vorne größer als 20 cm und nach links größer als 20 cm ist. In diesem Fall wird davon ausgegangen, dass links zuvor eine Wand existierte. Demzufolge sollte der Roboter dieser Wand weiterhin folgen, falls dies möglich ist. Dafür sind folgende Schritte erforderlich:

1. Abfrage des gespeicherten Messwerts des linken Ultraschallsensors über die Methode `getLastWallSample()`
2. Vorwärtsfahren um eine Distanz, die vom Zeitpunkt des Verlassens der Wand abhängig ist (die Distanz umso geringer ist, je länger der vorher bestimmte Zeitpunkt zurückliegt)
3. Drehung um 90° gegen den Uhrzeigersinn
4. Vorwärtsfahren in drei Schritten um jeweils 10 cm, um sicherzustellen, dass sich die Wand wieder links vom Roboter befindet
5. dabei wird vor jedem der 10 cm Schritte durch die Messwerte des vorderen Ultraschallsensors überprüft, ob sich vor dem Roboter ein Hindernis befindet

(c) links befindet sich eine Wand, vorne befindet sich eine Wand

Diese Bedingung ist erfüllt, wenn der Abstand nach vorne kleiner gleich 20 cm und nach links kleiner gleich 20 cm ist. In diesem Fall dreht sich der Roboter um 90° im Uhrzeigersinn, sodass sich die Wand wieder links von ihm befindet.

(d) links befindet sich keine Wand, vorne befindet sich eine Wand

Diese Bedingung ist erfüllt, wenn der Abstand nach vorne kleiner gleich 20 cm und nach links größer als 20 cm ist. In diesem Fall zieht er sich bis auf 10 cm an die vordere Wand heran. Anschließend dreht sich der Roboter um 90° gegen den Uhrzeigersinn, sodass er der verlorengegangenen Wand folgt, bis sie sich wieder links von ihm befindet. Dieser Fall ist insbesondere für Labyrinthe mit engen Gassen relevant, wie in Abschnitt 3.2.4 erkenntlich wird.

3.2.4 Demonstration

Um das entwickelte Projekt zur Navigation des Roboters durch das Labyrinth ausführen zu können, müssen mehrere Schritte unternommen werden. Zunächst muss die Datei die Datei `03_roboter_project/include/secrets.hpp` mit dem in Abschnitt 3.1.2 beschriebenen Inhalt angelegt werden, wenn dies noch nicht erfolgt ist. Außerdem wird ein Terminal geöffnet und in den ROS 2 Workspace (im Abgabeordner `03_roboter_project/ros2_ws`) gewechselt. Zusätzlich muss der Micro-ROS-Agent gestartet werden, wie in Abschnitt 3.1.2 beschrieben ist. Anschließend wird das Projekt mit dem Befehl

```
colcon build
```

gebaut. Danach muss das PIO-Projekt wie in Abschnitt 3.1.2 gebaut werden. Schließlich kann ein neues Terminal geöffnet und im Ordner des ROS 2 Workspaces mit

```
source install/setup.bash
ros2 run algorithm robot_controller_main
```

das Projekt gestartet werden.

Im Abgabeordner befindet sich unter `03_roboter_project/demo_algorithm.mp4` ein Video, durch das die korrekte Funktionsweise des implementierten Algorithmus demonstriert wird. Dieses Video musste stark komprimiert werden, weswegen die Aufnahme unscharf ist. Dadurch kann man beispielsweise die Ausgabetexte auf dem OLED-Display nur verschwommen sehen. Die relevanten Informationen sind trotzdem sihtbar. Da kein Audio vorliegt, kann man ebenso das Piepen des Buzzers während der Drehung nicht hören. Es wurde ein Labyrinth aufgebaut und aufgenommen, wie sich der Roboter entsprechend der Beschreibungen aus Abschnitt 3.2.3 durch das Labyrinth navigiert. Hierbei musste insbesondere darauf geachtet werden, dass der die Wege im Labyrinth breit genug sind. Außerdem müssen die Wände breit genug und lückenlos sein, damit sie korrekt erkannt werden. Die maximale Entfernung zu einer Wand nach vorne oder links darf maximal 2 Meter betragen, da der Ultraschallsensor nur diese maximale Entfernung messen kann. Bei größeren Abständen als 2 Metern wird eine 0 als Messwert veröffentlicht. Das Ziel wurde präpariert, indem ein Hindernis gewählt wurde, welches nicht hoch genug ist, um vom vorderen Ultraschallsensor registriert zu werden. Demzufolge kollidiert der Roboter mit diesem Hindernis. Durch die Aktivierung der Bumper fährt der Roboter anschließend rückwärts und beendet die Navigation.

4 Programmieraufgaben Echtzeitsysteme

In diesem Kapitel werden die implementierten Lösungen für die Programmieraufgaben [8] im Themenbereich Echtzeitsysteme dokumentiert. Zentrales Ziel ist dabei die Umsetzung grundlegender Mechanismen der Netzwerkkommunikation mithilfe von Sockets und der Interprozesskommunikation (IPC) in der Programmiersprache C. Der entwickelte Quellcode befindet sich im Verzeichnis `04_programming_exercises`. Für jede in den folgenden Abschnitten beschriebenen Teilaufgaben existiert ein Unterordner mit der zugehörigen Kapitelnummer. Darin befindet sich das `Makefile`, die Header-Dateien im Verzeichnis `include` und die Quelldateien im Verzeichnis `src`. Beim Ausführen des `Makefile` wird außerdem ein `build` Ordner erstellt, in dem die ausführbaren Dateien liegen.

Zusätzlich ist zu beachten, dass für die Verarbeitung Nachrichten im JSON-Format verwendet werden. Für die erfolgreiche Kompilierung der Programme ist deshalb die Bibliothek `cJSON` benötigt. Unter Ubuntu kann die Installation mit `sudo apt install libcjson-dev` erfolgen. Die konkreten Lösungen der einzelnen Teilaufgaben werden in den folgenden Abschnitten beschrieben.

4.1 Sockets und Netzwerkkommunikation

Der erste große Teilbereich der Programmieraufgaben bezieht sich auf die Netzwerkkommunikation mithilfe von Sockets. Dafür wird in Abschnitt 4.1.1 ein Server und in Abschnitt 4.1.2 der dazugehörige Client implementiert. Abschnitt 4.1.3 beinhaltet die Umsetzung der Verwaltung einer gemeinsamen Liste für mehrere Clients, die mit demselben Server kommunizieren. Abschließend wird in Abschnitt 4.1.4 beschrieben, wie dieses Konzept auf die Peer-to-Peer-Kommunikation erweitert werden kann. Abschließend wird in Tabelle 4.1 zusammengefasst, welche Aktionen aus dem Schließen einzelner Teilnehmer (Server, Client oder Peer) für die einzelnen Teilaufgaben resultieren.

4.1.1 Server

Es wurde ein TCP-Server mit nichtblockierenden Sockets implementiert, der eingehende Verbindungen annimmt, Anfragen verarbeitet und entsprechende Antworten an die Clients zurücksendet. Es wurde sich bewusst für nichtblockierende Sockets entschieden, damit nicht auf einen einzelnen Client gewartet werden muss, sondern die verfügbaren Client-Sockets fortlaufend überprüft werden können. Die für diese Teilaufgabe relevanten Dateien befinden sich im Verzeichnis `4_1_1_server`.

Als Anwendungsbeispiel wurde ein Taschenrechner umgesetzt, mit dem die arithmetischen Grundoperationen ausgeführt werden können. Dabei führt der Server die Berechnungen aus, die ihm vom Client übermittelt werden. Die Quelldatei `src/server.c` enthält die Hauptfunktion und steuert den Ablauf der Server-Anwendung. Die Implementierung orientiert sich an den in [9] gegebenen Beispieldateien. Die Client-Sockets werden in einem Array verwaltet, dessen maximale Größe durch `MAX_CLIENT_COUNT` festgelegt ist. In der Hauptschleife nimmt der Server neue Verbindungen entgegen und prüft anschließend die vorhandenen Client-Sockets auf eingehende Daten. Empfangene Nachrichten werden zunächst als Zeichenkette gespeichert und danach mit cJSON geparst. Nach der Verarbeitung der Anfrage erstellt der Server eine JSON-Antwort und sendet diese über den zugehörigen Client-Socket zurück. Wird eine Verbindung geschlossen, schließt den entsprechenden Socket und gibt den Eintrag im Client-Array wieder frei.

Für den strukturierten Austausch der Anfragen und Antworten wird ein eigenes Kommunikationsprotokoll implementiert. Das Kommunikationsprotokoll definiert den Aufbau der zwischen Client und Server ausgetauschten Protokolldateneinheiten (PDUs). Sowohl Anfragen als auch Antworten bestehen aus einem JSON-Objekt mit den beiden Hauptfeldern `header` und `payload`. Der `header` enthält die Metadaten der Nachricht. Dabei dient das Feld `msg_id` als Nachrichtenkennung, wobei der Wert bei jeder Anfrage um 1 erhöht wird. Das Feld `time_stamp` beschreibt den Zeitstempel der Nachricht in Sekunden. Dieser Zeitstempel wird zum Zeitpunkt der Generierung der Anfrage erstellt. Das Feld `payload` enthält die Nutzdaten. Dazu zählen die Felder `operation` und `operands`. Dabei ist `operation` die auszuführende Rechenoperation als Zeichenkette, wobei die Addition (+), Subtraktion (-), Multiplikation (*) und Division (/) unterstützt werden. Das Feld `operands` ist ein Array mit mindestens zwei Gleitkommazahlen. Eine beispielhafte Anfrage vom Client an den Server lautet:

```
{
  "header": {
    "msg_id": 1,
    "time_stamp": 1792853515
  },
  "payload": {
    "operation": "+",
    "operands": [783.099223758606,798.44003347607327,911.64735793678426]
  }
}
```

Der Server überprüft zunächst mithilfe der Datei `src/pdu_parsing.c`, ob die empfangene Nachricht in gültigem JSON-Format vorliegt und alle erforderlichen Felder vorhanden sind. Darüber hinaus werden die Datentypen der einzelnen Felder auf Korrektheit kontrolliert.

Die Berechnung des Ergebnisses auf Grundlage der Anfrage erfolgt durch die Quelldatei `calculator.c`. Zunächst wird geprüft, ob die angegebene Operation unterstützt wird. Eine nicht unterstützte Rechenoperation führt zu einem Fehlerstatus. Sind die Operation und das vorliegende Operanden-Array gültig, werden die angegebenen Operanden nacheinander mit der ausgewählten Operation verarbeitet. Bei der Division wird zusätzlich geprüft, ob einer der Divisoren den Wert null besitzt. Eine Division durch null wird als Fehler behandelt.

Anschließend wird die JSON-Antwort des Servers an den Client generiert. Die Einträge `msg_id` und `time_stamp` im `header` bleiben im Vergleich zur Anfrage im selben Format. Die Werte dieser Felder werden allerdings nicht aus der Anfrage des Clients übernommen. Der Wert für die `msg_id` wird im Server separat verwaltet und bei jeder erzeugten Antwort an einen Client um 1 erhöht. Der Zeitstempel wird zum Zeitpunkt der Generierung der Antwort an den Client erstellt. Der `payload` wird allerdings anders gestaltet. Dieser enthält nur noch das Feld `result`. Bei erfolgreicher Berechnung wird darin das Ergebnis der durchgeführten Berechnung eingetragen. Bei einem Berechnungsfehler wird stattdessen die Zeichenkette `NULL` übertragen. Damit unterscheidet das Protokoll zwischen einer erfolgreich ausgeführten Operation und einem Fehler während der Berechnung. Das Format der Antwort des Servers wird anhand der folgenden Beispielnachricht veranschaulicht:

```
{
  "header": {
    "msg_id": 1,
    "time_stamp": 1792853515
  },
  "payload": {
    "result": 2493.1866151714635
  }
}
```

Um dieses Projekt ausführen zu können, muss zunächst im Terminal in den Ordner `4_1_1_server` gewechselt und der Befehl `make` eingegeben werden. Anschließend wird der Ordner `build` erstellt. In diesen kann gewechselt und der Server mit `./server` gestartet werden. Der Ablauf zum Starten von Clients wird im folgenden Abschnitt beschrieben.

4.1.2 Client

Die Client-Anwendung ermöglicht die Kommunikation mit dem in Abschnitt 4.1.1 beschriebenen Server. Der Client baut eine TCP-Verbindung auf, sendet Anfragen an den Server und empfängt anschließend die entsprechenden Antworten. Die Implementierung befindet sich im Verzeichnis `4_1_2_client`.

Die Quelldatei `src/client.c` enthält die Hauptfunktion und steuert den Ablauf der Client-Anwendung. Nach erfolgreichem Verbindungsaufbau zum Server werden fortlaufend Anfragen an den Server gesendet und Antworten vom Server verarbeitet. Die Erzeugung der Anfragen übernimmt die Quelldatei `src/random_request_generator.c`. Darin wird ein JSON-Objekt mit den beiden Hauptfeldern `header` und `payload` erstellt. Im `header` werden die `msg_id` sowie ein zur Laufzeit erstellter Zeitstempel `time_stamp` eingetragen. Im `payload` werden die auszuführende Rechenoperation und die Operanden festgelegt. Die Rechenoperation wird zufällig aus den vier unterstützten Operationen Addition (+), Subtraktion (-), Multiplikation (*) und Division (/) ausgewählt. Außerdem wird eine zufällige Anzahl von zwei bis fünf Operanden bestimmt. Die einzelnen Operanden sind zufällig generierte Gleitkommazahlen zwischen 0 und 1000. Ein beispielhaftes Format dieser generierten JSON-Nachricht ist in Abschnitt 4.1.1 beschrieben. Die erzeugte JSON-Anfrage wird über den TCP-Socket an den Server gesendet. Dabei überprüft der Client, ob beim Senden ein Fehler aufgetreten ist oder nicht die gesamte Nachricht übertragen wurde. In beiden Fällen wird eine Fehlermeldung ausgegeben, der Socket geschlossen und die Client-Anwendung beendet. Nach dem Senden wartet der Client auf eine Antwort des Servers. Wenn keine Antwort vom Server empfangen wird, wird die Verbindung vom Server geschlossen. Bei einem Empfangsfehler wird eine Fehlermeldung ausgegeben und die Client-Anwendung beendet. Falls eine Antwort vom Server eingegangen ist, werden die empfangenen Daten in einem Puffer gespeichert. Anschließend wartet der Client zehn Sekunden, bis er eine neue Anfrage generiert und somit die Schleife von vorne startet.

Für die Ausführung des Projekts muss zunächst in das Verzeichnis `4_1_2_client` gewechselt und das Programm mit `make` kompiliert werden. Anschließend kann der Client aus dem Verzeichnis `build` mit `./client` gestartet werden. Die Voraussetzung dafür ist, dass der Server (wie in Abschnitt 4.1.1 beschrieben) bereits gestartet wurde. Es können weitere Clients gestartet werden, indem dieses Vorgehen in mehreren Terminals ausgeführt wird.

4.1.3 Verwaltung einer gemeinsamen Liste

In dieser Teilaufgabe wird auf dem Server eine Liste von Nachrichtenverläufen angelegt, auf die mehrere Clients zugreifen können. Die Clients sollen bestehende Nachrichtenverläufe abfragen, neue Nachrichten hinzufügen und vorhandene Nachrichten bearbeiten können. Die Implementierung befindet sich im Verzeichnis `4_1_3_shared_chats`.

Die Nachrichtenverläufe sind in der Quelldatei `src/server.c` deklariert. Das dreidimensionale Array `chats[MAX_CHAT_COUNT][MAX_MSG_COUNT][MAX_MSG_LENGTH]` umfasst `MAX_CHAT_COUNT` Chats mit jeweils bis zu `MAX_MSG_COUNT` Nachrichten. Jede Nachricht wird als Zeichenkette mit maximal `MAX_MSG_LENGTH-1` Zeichen und einem abschließenden Nullzeichen gespeichert. Beim Start des Servers werden in `src/server_utils.c` die ersten 20 Chats mit Beispiel-Nachrichtenverläufen initialisiert, während die anderen 80 Chats leer bleiben. Diese Chats werden ausschließlich vom Server verwaltet und können von allen verbundenen Clients abgefragt und geändert werden.

Die Kommunikation zwischen Server und Client basiert weiterhin auf JSON-Nachrichten mit den Hauptfeldern `header` und `payload`. Der `header` enthält die Nachrichtenkennung `json_msg_id` und den Zeitstempel `time_stamp` in Sekunden. Für die Anfragen vom Client an den Server wurden 4 verschiedene Operationen implementiert, die weiter unten beschrieben werden. Von dieser Operation ist somit die konkrete Struktur des Felds `payload` der JSON-Nachricht abhängig, sowohl für die Anfrage des Clients an den Server als auch für die Antwort des Servers an den Client. Das Feld `operation` gibt dabei die Art der Operation an und ist deshalb unabhängig von der Operation im `payload` enthalten.

Die Anfrage des Clients wird vom Server in der Hauptschleife der Datei `src/server.c` verarbeitet. Dazu wird zunächst das korrekte Format der geparsten Nachricht mithilfe von `src/pdu_parsing.c` geprüft. Dazu zählen die Angabe der notwendigen Felder und die Einhaltung der Datentypen. Außerdem wird geprüft, ob die angegebene `chat_id` oder `msg_id` im gültigen Wertebereich von 0 bis `MAX_CHAT_COUNT` bzw. `MAX_MSG_COUNT` liegen oder Nachrichteninhalte eine Länge von 1 bis `MAX_MSG_LENGTH-1` haben. In Abhängigkeit von der Operation der Anfrage generiert der Server die JSON-Antwort für den Client. Dabei wird im `payload` mit dem Feld `success` übermittelt, ob die Anfrage erfolgreich verarbeitet werden konnte. Ist dies der Fall, enthält dieses Feld den Wert 0. Sonst ist der Wert ungleich 0 und die Fehlerbeschreibung wird im Feld `failure_reason` angegeben.

In der Quelldatei `random_request_generator.c` sind Funktionen zur Generierung der JSON-Anfragen für folgenden 4 unterschiedliche Operationen implementiert:

1. Mit der Operation `get_chat` fragt der Client den Nachrichtenverlauf des Chats mit der übergebenen `chat_id` an. Eine beispielhafte JSON-Abfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792923496
  },
  "payload": {
    "operation": "get_chat",
    "chat_id": 13
  }
}
```

Der Server bearbeitet diese Anfrage und sendet die Chat-ID, den Nachrichtenverlauf für den angeforderten Chat als Array von Nachrichten sowie die Erfolgsmeldung zurück an den Client. Eine beispielhafte Antwort auf die beschriebene Anfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792938326
  },
  "payload": {
    "chat_id": 13,
    "content": ["Hast du einen USB-Stick?","Ja, hier.","Danke."],
    "success": 0
  }
}
```

2. Mit der Operation `get_msg` fragt der Client eine einzelne Nachricht eines Chats vom Server an. Dementsprechend müssen die `chat_id` und die `msg_id` in der Anfrage enthalten sein. Eine beispielhafte JSON-Abfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792923496
  },
  "payload": {
    "operation": "get_msg",
    "chat_id": 13,
    "msg_id": 1
  }
}
```

Der Server bearbeitet diese Anfrage und sendet die Chat-ID, die Message-ID, den Inhalt der angeforderten Nachricht sowie die Erfolgsmeldung zurück an den Client. Eine beispielhafte Antwort auf die beschriebene Anfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792938326
  },
  "payload": {
    "chat_id": 13,
    "msg_id": 1,
    "content": "Ja, hier.",
    "success": 0
  }
}
```

3. Mit der Operation `add_msg` sendet der Client eine Anfrage an den Server, um eine Nachricht zu einem Chat hinzuzufügen. Dementsprechend müssen die `chat_id` sowie der Inhalt der hinzuzufügenden Nachricht in der Anfrage enthalten sein. Die `msg_id` ist nicht erforderlich, da sie anhand der bisherigen Länge des angefragten Chats serverseitig berechnet wird. Der Inhalt der Nachricht wird dabei zufällig generiert. Dafür wird zufällig eine Länge von 1 bis 99 ausgewählt und der Inhalt mit zufälligen Klein-, Großbuchstaben und Ziffern gefüllt. Eine beispielhafte JSON-Abfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792923496
  },
  "payload": {
    "operation": "add_msg",
    "chat_id": 13,
    "content": "6NYRE3Md5oLUI8Qju"
  }
}
```

   Der Server bearbeitet diese Anfrage und sendet die Chat-ID, die Message-ID, eine Beschreibung der durchgeführten Aktion sowie die Erfolgsmeldung zurück an den Client. Das Format der Antwort des Servers wird anhand der folgenden Beispielnachricht veranschaulicht:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792938326
  },
  "payload": {
    "chat_id": 13,
    "msg_id": 3,
    "remark": "Message added as msg with id 3 in chat with id 13",
    "success": 0
  }
}
```

4. Mit der Operation `edit_msg` sendet der Client eine Anfrage an den Server, um den Inhalt einer Nachricht aus einem Chat zu bearbeiten. Dementsprechend muss die `chat_id`, die `msg_id` sowie der Inhalt der zu bearbeitenden Nachricht in der Anfrage enthalten sein. Der Inhalt der Nachricht wird dabei zufällig generiert. Dafür wird zufällig eine Länge von 1 bis `MAX_MSG_LENGTH-1` ausgewählt und der Inhalt mit zufälligen Klein-, Großbuchstaben und Ziffern gefüllt. Eine beispielhafte JSON-Abfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792923496
  },
  "payload": {
    "operation": "edit_msg",
    "chat_id": 13,
    "msg_id": 1,
    "content": "CiPTzdzboItSE6EjyxTBq6aaE7QLrxWTFBA2EY3TwkzYe37DoP2"
  }
}
```

Der Server bearbeitet diese Anfrage und sendet die Chat-ID, die Message-ID, eine Beschreibung der durchgeführten Aktion sowie die Erfolgsmeldung zurück an den Client. Eine beispielhafte Antwort auf die beschriebene Anfrage hat das Format:

```
{
  "header": {
    "json_msg_id": 1,
    "time_stamp": 1792938326
  },
  "payload": {
    "chat_id": 13,
    "msg_id": 1,
    "remark": "Edited msg with id 1 in chat with id 13",
    "success": 0
  }
}
```

Hierbei ist anzumerken, dass der in der Quelldatei `src/client.c` implementierte Ablauf für die Generierung der Anfragen folgender ist:

1. Es wird ein zufälliger Chat mit `get_chat` abgefragt.
2. Die Antwort des Servers wird geparst, um die Länge des zurückgegebenen Nachrichtenverlaufs zu bestimmen.
3. Es wird eine Nachricht zum in Schritt 1 angefragten Chat mit `add_msg` hinzugefügt.
4. Mithilfe von `get_chat` wird der oben genannte Chat erneut vom Server angefordert. Anschließend wird geprüft, ob die in Schritt 2 bestimmte Länge des Nachrichtenverlaufs größer als 0 ist. Ist dies der Fall, wird mit Schritt 5 fortgefahren. Andernfalls beginnt der Client wieder bei Schritt 1.
5. Der Inhalt einer Nachricht des in Schritt 1 angefragten Chats wird mit `edit_msg` bearbeitet. Die zu bearbeitende Nachricht wird dabei zufällig aus allen verfügbaren Nachrichten im Chat (siehe Schritt 2) ausgewählt. Demzufolge wird die in Schritt 3 hinzugefügte Nachricht auf keinen Fall ausgewählt.
6. Die bearbeitete Nachricht wird mithilfe von `get_msg` vom Server angefordert. Anschließend beginnt der Client wieder bei Schritt 1.

Für die Ausführung des Projekts muss zunächst in das Verzeichnis `4_1_3_shared_chats` gewechselt und das Programm mit `make` kompiliert werden. Anschließend können der Server sowie ein oder mehrere Clients aus dem Verzeichnis `build` mit `./server` bzw. `./client` gestartet werden.

4.1.4 Peer-to-Peer-Erweiterung

In dieser Teilaufgabe wurde die in Abschnitt 4.1.3 beschriebene Client-Server-Architektur zu einer Peer-to-Peer-Architektur erweitert. Während zuvor ein Client Anfragen an einen Server stellte, übernimmt nun jeder Peer beide Rollen gleichzeitig. Er stellt einen Server bereit, um Anfragen anderer Peers entgegenzunehmen und baut selbst ausgehende Verbindungen auf, um Anfragen an andere Peers zu senden. Darüber hinaus wurde die bisherige Verarbeitung um Synchronisationsmechanismen erweitert, sodass die Chats in allen verbundenen Peers übereinstimmen. Die Implementierung für diese Teilaufgabe befindet sich im Verzeichnis `4_1_4_peer`.

Die Initialisierung eines Peers erfolgt in der Datei `peer.c`. Nach dem Erstellen eines Peers versucht das Programm, Verbindungen zu anderen Peers aufzubauen. Die ausgehenden Verbindungen werden in `outgoing_peer_fds` verwaltet und die zugehörigen Serverports werden in `connected_peer_server_ports` gespeichert. Eingehende Verbindungen werden dagegen über `incoming_peer_fds` verwaltet. Die getrennte Verwaltung ist notwendig, da ein Peer gleichzeitig Verbindungen annehmen und selbst Verbindungen initiieren kann. Die einmalige Verbindungsinitialisierung wurde um eine wiederkehrende Peer-Erkennung erweitert. In der Hauptschleife werden die möglichen Serverports regelmäßig überprüft. Ist ein Peer noch nicht über eine ausgehende Verbindung erreichbar und existiert ein freier Eintrag in der Verbindungsliste, versucht das Programm erneut, eine Verbindung zu diesem Peer aufzubauen. Bereits bestehende Verbindungen werden dabei berücksichtigt, um doppelte ausgehende Verbindungen zum selben Serverport zu vermeiden.

Da jeder Peer seine eigenen Chats verwaltet, muss beim Verbinden eines neuen Peers zunächst ein gemeinsamer Ausgangszustand hergestellt werden. Dazu sendet der neue Peer eine Initialisierungsanfrage mit dem Feld `init_sync` an den ersten Peer, zu dem er eine Verbindung aufgebaut hat. Dieser antwortet mit einer Bestätigungs-Nachricht mit dem Feld `init_ack`, die seine aktuelle Chatliste im Feld `content` enthält. Der neue Peer übernimmt die empfangenen Chatdaten in seine eigene Chatliste und kann anschließend an der regulären Synchronisation teilnehmen. Da die Chatlisten der bereits verbundenen Peers durch die laufende Synchronisation auf demselben Stand gehalten werden sollen, ist grundsätzlich unerheblich, von welchem dieser Peers der neue Peer seine initialen Daten erhält.

Der prinzipielle Ablauf aus Abschnitt 4.1.3 wurde größtenteils beibehalten und auf die Kommunikation zwischen Peers übertragen. Der Ablauf der 6 Schritte zum Senden der Anfragen aus Abschnitt 4.1.3 ist erhalten geblieben. Allerdings erfolgt das Senden der Anfrage nun nicht mehr zum Server, sondern zu einem anderen verbundenen Peer im Netzwerk, der zufällig ausgewählt wird. Die Kommunikation wurde außerdem um spezielle Synchronisationsnachrichten und Bestätigungen erweitert. Diese werden anhand zusätzlicher Felder im JSON-Header erkannt. Das Feld `sync` kennzeichnet eine Synchronisationsnachricht, während das Feld `ack` zur Erkennung einer Bestätigung verwendet wird. Die jeweilige Kennzeichnung enthält den Serverport des sendenden Peers. Darüber hinaus enthält die Bestätigungsnachricht ein Feld `success`, welches im Fall einer erfolgreich verarbeiteten JSON-Synchronisationsnachricht 0 ist.

Die Synchronisationslogik wurde in der zusätzlichen Datei `peer_synchronization.c` zusammengefasst. Damit wird die bisherige Anfrageverarbeitung um Funktionen ergänzt, die Synchronisationsanfragen erzeugen, versenden und bestätigen. Ein Peer sendet dabei zunächst an sich selbst die Anfrage, die Nachrichten zu bearbeiten. Die Funktion `update_own_chats()` unterscheidet dabei zwischen dem Hinzufügen einer Nachricht zu einem bestehenden Chat mit `add_msg` und dem Bearbeiten einer vorhandenen Nachricht mit `edit_msg`. Anschließend wird eine passende Synchronisationsnachricht erzeugt, die nacheinander an alle anderen verbundenen Peers gesendet wird. Dadurch wird sichergestellt, dass alle verbundenen Peers zu jeder Zeit dieselben Nachrichtenverläufe gespeichert haben. Empfängt ein Peer eine Synchronisationsnachricht, werden die Chats dementsprechend aktualisiert. Danach erzeugt der empfangende Peer eine Bestätigung und sendet diese an den anfragenden Peer zurück.

Um den Synchronisationsablauf nachvollziehen zu können, wurde die Verwaltung der Peer-Verbindungen um Informationen zum Synchronisationsstatus ergänzt. Grundlegend werden gewöhnlichen Anfragen, Synchronisationsnachrichten und Bestätigungen unterschieden. Eine Bestätigung wird gesondert behandelt und nicht wie eine normale Chat-Anfrage verarbeitet. Eine Synchronisationsnachricht führt dagegen zur Verarbeitung der enthaltenen Operation und anschließend zur Erzeugung einer Bestätigung. In `peer.c` wird beim Versenden einer Synchronisationsnachricht festgehalten, welche Peer-Verbindungen kontaktiert wurden. Die entsprechenden Einträge werden in den Listen `contacted_peer_fds` und `pending_sync_ack_peer_indexes` verwaltet. Die erfolgreich synchronisierten Peers werden in der Liste `successfully_synchronized_peer_fds` festgehalten. Die Hauptschleife überprüft daraufhin, ob für alle aktuell berücksichtigten ausgehenden Verbindungen eine Bestätigung vorliegt und aktualisiert `sync_ack_pending`.

Für die Ausführung des Projekts muss zunächst in das Verzeichnis `4_1_4_peer` gewechselt und das Programm mit `make` kompiliert werden. Anschließend können mehrere Peers gestartet werden, beispielsweise mit `./peer 0`, `./peer 1` usw. Das zusätzliche Kommandozeilenargument `argv[1]` muss angegeben werden und für jeden Peer einen unterschiedlichen Wert haben, da sich der Server-Port aus `DEFAULT_PORT+argv[1]` ergibt.

Tabelle 4.1: Übersicht der Aktionen beim Trennen einzelner Teilnehmer

Teilaufgabe	**Beschreibung der Aktionen**
`4_1_1 + 4_1_2 + 4_1_3`	• wird ein Client vor dem Server gestartet, kann der Client keine Verbindung aufbauen und schließt sich direkt wieder • wird der Server beendet, schließen sich die verbundenen Clients direkt oder mit einer kurzen Verzögerung durch `sleep()` • das Beenden eines einzelnen Clients hat keine Auswirkungen auf den weiteren Ablauf, da alle anderen Clients weiterhin mit dem Server verbunden sind • nachdem der letzte verbundene Client beendet wurde, bleibt der Server aktiv und kehrt in seine Initialisierungsschleife zurück, wobei er auf die Verbindung von Clients wartet
`4_1_4`	• wird ein Peer beendet, können die übrigen Peers ihre Ausführung fortsetzen • nach dem Beenden des letzten Peers wird die Sitzung beendet • ein anschließendes Starten eines neuen Peers führt zum Beginn einer neuen Sitzung

4.2 IPC

Der zweite große Teilbereich der Programmieraufgaben bezieht sich auf die Interprozesskommunikation (IPC). Diese wurde mithilfe von Named Pipes (FIFOs) umgesetzt, anstatt beispielsweise Shared Memory oder Message Queues zu verwenden. Ausschlaggebend dafür war insbesondere die in Abschnitt 4.1 umgesetzte Kommunikation über TCP-Sockets, deren grundlegendes Prinzip der Datenübertragung sich auf FIFOs übertragen lässt. Dadurch kann die bestehende Verarbeitung von Anfragen und Antworten weitgehend beibehalten und mit geringem Anpassungsaufwand auf die Teilaufgaben in diesem Abschnitt übertragen werden. Für Shared Memory ist wäre außerdem ein gemeinsam genutzter Speicherbereich erforderlich. Für die FIFOs werden gemeinsame Dateien benötigt, die im Verzeichnis `04_programming_exercises/ipc_files` abgelegt sind. Eine Ausnahme bildet Aufgabe 4.2.4, in der die FIFOs im Unterverzeichnis `04_programming_exercises/ipc_files/fifos` gespeichert werden. Da die Programme Registrierungsdateien und FIFO-Dateien anlegen und verwenden, sollte vor jedem Programmstart sichergestellt werden, dass das Verzeichnis `ipc_files` leer ist oder zuvor gelöscht wird. Der Empfang und Versand der Nachrichten wurde auf `read()` und `write()` umgestellt. In den folgenden Abschnitten werden die Implementierungen beschrieben. Für die Aktionen nach dem Schließen einzelner Teilnehmer (Server, Client oder Peer) gilt weiterhin Tabelle 4.1, nur dass sich Clients direkt und ohne Wartezeit trennen. Außerdem warten verbunden Clients auf den Server und beenden sich nicht direkt.

4.2.1 Server-Prozess

Für diese Teilaufgabe bleiben die grundlegende Verarbeitung der Anfragen und die anschließende Berechnung des Ergebnisses für die Kommunikation über FIFOs im Vergleich zu Abschnitt 4.1.1 unverändert. Die Implementierung für diese Teilaufgabe befindet sich im Verzeichnis `4_2_1_server_process`. Die durchgeführten Anpassungen für diese Teilaufgabe befinden sich hauptsächlich in der Quelldatei `src/server.c`.

Für die Übertragung der Anfragen vom Client zum Server wird ein gemeinsamer FIFO mit dem Namen `c2s` verwendet. Für die Antworten wird dagegen für jeden Client eine separater FIFO mit dem Namen `s2c<PID>` angelegt, wobei `PID` die Prozess-ID des jeweiligen Clients ist. Der jeweilige Client bildet das lesende Ende, während der Server in diesen FIFO schreibt. Dadurch kann der Server anhand der mit der Anfrage übermittelten PID den zugehörigen Antwort-FIFO auswählen und die Antwort gezielt an den anfragenden Client zurücksenden. Die Trennung ist notwendig, damit Clients keine Antworten empfangen, die für andere Clients bestimmt sind.

Der Server muss die vorhandenen FIFOs selbstständig erkennen. Hierfür wird das Verzeichnis `ipc_files` nach FIFO-Dateien durchsucht. Außerdem werden während der Laufzeit neu hinzugekommene Clients erkannt. Die zugehörigen Dateipfade und geöffneten Dateideskriptoren werden in Arrays verwaltet

Zum Starten des Server-Prozesses können dieselben Schritte wie in Abschnitt 4.1.1 durchgeführt werden, allerdings angepasst auf das Verzeichnis `4_2_1_server_process`.

4.2.2 Client-Prozess

Die Implementierung der Kommunikation über FIFOs für diese Teilaufgabe befindet sich im Verzeichnis `4_2_2_client_process`, wobei die im Vergleich zu Abschnitt 4.1.2 vorgenommenen Änderungen hauptsächlich die Quelldatei `src/client.c` betreffen.

Zu Beginn ermittelt der Client seine Prozess-ID (PID), die zur eindeutigen Zuordnung der Kommunikation verwendet wird. Anschließend werden das Verzeichnis `ipc_files` und die benötigten FIFOs angelegt, sofern diese noch nicht existieren. Für das Senden der Anfragen wird der FIFO `c2s` geöffnet, während Antworten über ein eigenes FIFO `s2c<PID>` empfangen werden, wie bereits in Abschnitt 4.2.1 beschrieben wurde. Die Prozess-ID wird zusätzlich im Header der Anfrage im Feld `client_pid` übertragen, damit der Server die Antwort der richtigen FIFO zuordnen kann. Die Generierung und der Ablauf der Anfragen bleiben im Vergleich zu den Beschreibungen aus Abschnitt 4.1.2 unverändert.

Ein Unterschied zur in Abschnitt 4.1.2 beschriebenen Implementierung ist die Behandlung von Unterbrechungen der Kommunikation. Tritt beim Schreiben ein Fehler auf, weil kein Server die Anfrage-FIFO geöffnet hat, versucht der Client, die Verbindung durch erneutes Öffnen des FIFO wiederherzustellen. Wird beim Lesen festgestellt, dass der Antwort-FIFO geschlossen wurde, versucht der Client ebenfalls, diese erneut zu öffnen. Über einen Signal-Handler wird außerdem ein Tastaturabbruch mit `SIGINT` erkannt. Beim Beenden des Programms schließt der Client seine Dateideskriptoren und entfernt seinen eigenen Antwort-FIFO.

Zum Starten des Client-Prozesses können dieselben Schritte wie in Abschnitt 4.1.2 durchgeführt werden, allerdings angepasst auf das Verzeichnis `4_2_2_client_process`.

4.2.3 Verwaltung einer gemeinsamen Liste

Die Verwaltung der gemeinsamen Liste und die unterstützten Operationen bleiben bei der Kommunikation über FIFOs im Vergleich zu Abschnitt 4.1.3 unverändert. Die Implementierung für diese Teilaufgabe befindet sich im Verzeichnis `4_2_3_shared_chats_ipc`. Die Anpassungen betreffen hauptsächlich die Übertragung der Nachrichten und die Zuordnung der Clients.

Das Array `chats[MAX_CHAT_COUNT][MAX_MSG_COUNT][MAX_MSG_LENGTH]` wird wie in Abschnitt 4.1.3 im Server verwendet, um die Nachrichtenverläufe zu speichern. Zum Empfang der Client-Anfragen wird der FIFO `c2s` verwendet. Die Antworten werden über separate FIFOs mit dem Namen `s2c<PID>` an die jeweiligen Clients übertragen. Dazu enthält jede Anfrage im Header neben der Nachrichtenkennung `json_msg_id` und dem Zeitstempel `time_stamp` zusätzlich die Prozess-ID `client_pid`. Der Server ermittelt anhand dieser Prozess-ID den zugehörigen FIFO-Pfad und den entsprechenden Dateideskriptor.

Die Fehlerbehandlung wurde an die FIFO-Kommunikation angepasst. Dafür erzeugt die Funktion `error_function_with_response()` bei erkannten Fehlern eine entsprechende JSON-Fehlerantwort und überträgt diese über den Dateideskriptor des jeweiligen Antwort-FIFOs. Die Antwort enthält weiterhin das Feld `success` mit einem Fehlercode (0 für Erfolg, sonst ungleich 0) sowie das Feld `failure_reason` mit einer Beschreibung des Fehlers.

Der Ablauf des Clients wurde dahingehend angepasst, dass nach dem Abfragen eines Chats zunächst eine Nachricht hinzugefügt und anschließend geprüft wird, ob sich der Nachrichtenverlauf entsprechend verändert hat. Ist der Chat nicht leer, wird eine vorhandene Nachricht bearbeitet und anschließend mit `get_msg` erneut abgefragt, um die Änderung zu überprüfen.

Für die Ausführung des Projekts können dieselben Schritte wie in Abschnitt 4.1.3 durchgeführt werden, allerdings angepasst auf das Verzeichnis `4_2_3_shared_chats_ipc`.

4.2.4 Gleichberechtigte Kommunikation

Im Gegensatz zur Client-Server-Architektur aus Abschnitt 4.2.3 verwaltet jeder Peer seine eigene Kopie der gemeinsamen Chatliste. Die Implementierung für diese Teilaufgabe befindet sich im Verzeichnis `4_2_4_equal_communication` und umfasst insbesondere die Erkennung neuer Peers, den Aufbau der FIFO-Verbindungen sowie die Initialisierung und Synchronisation der Chatlisten. Das grundlegend übernommene Konzept ist in Abschnitt 4.1.4 beschrieben.

Für die Identifikation der Peers wird deren Prozess-ID (PID) verwendet. Beim Start erstellt jeder Peer eine Registrierungsdatei mit seiner PID im Verzeichnis `ipc_files/peers`. Die FIFO-Dateien werden im Unterverzeichnis `04_programming_exercises/ipc_files/fifos` abgelegt. Anhand der Registrierungsdateien können die Peers erkennen, welche anderen Peers verfügbar sind und entsprechende Kommunikationsverbindungen aufbauen.

Um die Nachrichten zwischen jedem Paar von Peers übertragen zu können, werden für jedes dieser Peer-Paare zwei FIFOs mit dem Namen `PIDWritingPeer_2_PIDReadingPeer` angelegt. Beispielsweise verwendet Peer 1 mit PID 1001 zum Senden an Peer 2 mit PID 1002 den FIFO mit dem Namen `1001_2_1002`. Für die Übertragung in die Gegenrichtung wird der FIFO mit dem Namen `1002_2_1001` verwendet. Jeder FIFO wird auf der einen Seite zum Schreiben und auf der anderen Seite zum Lesen geöffnet. Dadurch kann jeder Peer sowohl Anfragen an andere Peers senden als auch deren Nachrichten empfangen. Für den Empfang werden die eingehenden FIFOs nichtblockierend geöffnet. Mithilfe von `poll()` wird anschließend nichtblockierend geprüft, auf welchen FIFOs neue Daten verfügbar sind, sodass Nachrichten verschiedener Peers verarbeitet werden können, ohne für jeden Peer einen separaten Empfangsprozess zu benötigen.

Da jeder Peer seine eigene Chatliste verwaltet, muss beim Beitritt eines neuen Peers zunächst ein gemeinsamer Ausgangszustand hergestellt werden. Dazu sendet der neue Peer eine Initialisierungsanfrage (`init_sync`) an den ersten Peer, zu dem er eine Verbindung aufgebaut hat. Dieser antwortet mit einer Bestätigungs-Nachricht (`init_ack`), die seine aktuelle Chatliste im Feld `content` enthält. Der neue Peer übernimmt die empfangenen Chatdaten in seine eigene Chatliste und kann anschließend an der regulären Synchronisation teilnehmen. Da die Chatlisten der bereits verbundenen Peers durch die laufende Synchronisation auf demselben Stand gehalten werden sollen, ist grundsätzlich unerheblich, von welchem dieser Peers der neue Peer seine initialen Daten erhält.

Während des laufenden Betriebs erfolgt die Synchronisation in mehreren Schritten. Zunächst führt ein Peer eine Änderung an seiner eigenen Chatliste durch. Anschließend generiert er eine Synchronisationsnachricht, die alle Informationen enthält, die auch für das Update der eigenen Variable erforderlich waren. Zudem wird eine PID mitgesendet, damit der empfangende Peer weiß, an welchen Peer er eine Bestätigungsnachricht senden muss. Der Peer sendet die generierte Synchronisationsnachricht anschließend an die verbundenen Peers. Die empfangenden Peers prüfen die Nachricht, führen die angeforderte Operation auf ihrer eigenen Chatliste aus und senden anschließend eine Bestätigungsnachricht (`ack`) zurück. Diese enthält unter anderem eine Erfolgsinformation im Feld `success`, welches im Fall einer erfolgreich verarbeitenden JSON-Synchronisationsanfrage 0 ist. Weiterhin schreibt der sendende Peer seine eigene PID in die Bestätigungsnachricht, damit der empfangende Peer weiß, welche Peers die Synchronisationsoperation ausgeführt haben. Somit kann der ursprüngliche Peer anhand der Bestätigungen nachvollziehen, ob die Verarbeitung bei den adressierten Peers erfolgreich war.

Eingehende Nachrichten werden erst geparst und auf ihre grundlegende JSON-Struktur geprüft. Anschließend werden die Header- und Payload-Felder geprüft, die bei jeder Operation gesendet werden. Entsprechend des Nachrichtentyps (bspw. `sync`) und der angefragten Operation werden die für diese Operation erforderten Felder überprüft. Dazu gehören bspw. das Abfragen eines Chats, das Hinzufügen einer Nachricht und die anschließende Überprüfung des geänderten Chatverlaufs. Bei der Bearbeitung einer Nachricht wird die Änderung ebenfalls durch eine nachfolgende Abfrage überprüft. Die Reihenfolge dieser Operationen wird über eine Workflow-Verwaltung gesteuert. Dadurch lässt sich nachvollziehen, ob die ausgeführte Änderung auch im synchronisierten Datenbestand vorliegt.

Die Peer-to-Peer-Architektur ermöglicht eine dezentrale Verwaltung der Chatlisten, bei der jeder Peer sowohl Anfragen verarbeitet als auch Änderungen an andere Peers weitergibt. Die Initialisierung stellt beim Beitritt eines neuen Peers einen gemeinsamen Ausgangszustand her. Die anschließende Übertragung von Änderungen und die Rückmeldungen der empfangenden Peers dienen dazu, die Chatlisten während des laufenden Betriebs konsistent zu halten und Fehler zu überprüfen.

Außerdem wurde ein Mechanismus programmiert, der sicherstellt, dass beim Verbindungsabbau einzelner Peers die zugehörigen Peer-Dateien im `ipc_files/peers`-Verzeichnis gelöscht werden und die geöffneten FIFOs geschlossen werden, damit die anderen Peers weiter korrekt funktionieren. Die Discovery ermöglicht weiterhin, den Verbindungsabbau verbundener Peers zu erkennen.

Für die Ausführung des Projekts können dieselben Schritte wie in Abschnitt 4.1.4 durchgeführt werden, allerdings angepasst auf das Verzeichnis `4_2_4_equal_communication`.

5 Zusatzaufgaben

Im Rahmen der Zusatzaufgaben wurde sich mit FreeRTOS beschäftigt, einem minimalistischen Betriebssystem. Das Ziel der Aufgaben ist die Nutzung des Task-Schedulers, sodass explizit die Core-Zurodnung von Tasks eingestellt werden kann und das Programm vom klassishcen Super Loop abweichen kann. Der Code für die Zusatzaufgaben befindet sich in `05_additional_tasks`. Wie vorgeschlagen, wurden die Videos aus [10] verwendet und der Code aus dieser Quelle angepasst, da die Videoreihe die gestellten Aufgaben behandelt.

5.1 Zusatzaufgabe 1

In der ersten Zusatzaufgabe wurde die LED an Pin 15 zum Blinken gebracht. Die Implementierung befindet sich unter `05_additional_tasks/additional_task1`. Dafür werden im globalen Teil der Quelldatei `src/main.cpp` die Variable `led_core` sowie die Funktion `toggleLED()` zum Blinken der LED erstellt. Die `toggleLED()`-Funktion zwischen den Toggles des LED-Zustands verwendet nicht die `delay()`-Funktion, sondern die `vTaskDelay()`-Funktion, damit CPU-Kern 0 nicht blockiert.

Weiterhin wurde ein Buzzer in Intervallen von einer Sekunde getoggelt. Für diesen wurde die Funktion `toggleBuzzer()`erstellt, die analog zur LED-Funktion läuft, nur das hier Buzzer-Funktionen verwendet werden. Weiterhin wird die Variable `buzzer_core` verwendet, die später zur Zuweisung des Tasks zu einem Core verwenet wird.

Im Setup-Teil wird der GPIO-Pin der LED als Ausgang gesetzt. Anschließend wird die Funktion `xTaskCreatePinnedToCore` verwendet, um die Funktion `toggleLED()` auf CPU-Kern 0 zu starten. Weiterhin wird das Setup des Buzzers durchgeführt und die Funktion `xTaskCreatePinnedToCore` verwendet, um die Funktion `toggleBuzzer()` auf CPU-Kern 1 auszuführen.

Der Loop-Teil enthält keinen relevanten Inhalt, da die auszuführenden Funktionen jeweils schon durch `xTaskCreatePinnedToCore` gestartet werden. Der Loop besteht aber aus einem vTaskDelay()-Aufruf, damit CPU-Kern 1 (auf dem der Loop läuft) nicht im Busy Wait-Zustand ist.

In der seriellen Ausgabe kann man erkennen, dass die Funktionen jeweils auf dem geplanten CPU-Kern laufen.

Die ursprüngliche Aufgabenstellung fordert lediglich, dass die LED-Task auf CPU-Kern 0 ausgeführt wird, während die normale Arduino-loop() auf CPU-Kern 1 verbleibt. Funktional äquivalent hierzu wäre die Umsetzung des Buzzers innerhalb der `loop()`-Funktion:

```
void loop() {
buzzer_on();
delay(1000);
Serial.printf("Core of buzzer task: %d\n", xPortGetCoreID());

buzzer_off();
delay(1000);
Serial.printf("Core of buzzer task: %d\n", xPortGetCoreID());
}
```

Die Auswertung der Konsole zeigt, dass die `loop()`-Funktion auf CPU-Kern 1 ausgeführt wird. Somit ist nachgewiesen, dass die Implementierung des Buzzers innerhalb der `loop()`-Funktion hinsichtlich der Core-Zuordnung äquivalent zur abgegebenen Lösung ist.

Die explizite Nutzung von `xTaskCreatePinnedToCore()` zum Pinnen des Buzzer-Tasks auf CPU-Kern 1 über RTOS geschieht somit im Quellcode und nicht implizit über die `loop()`-Funktion.

5.2 Zusatzaufgabe 2

Die zweite Zusatzaufgabe im Verzeichnis `05_additional_tasks/additional_task2/src` teilt sich in 2 Teilaufgaben:

1. Demonstration eines Ressourcenkonflikts in `main_with_resource_conflicts.cpp`
2. Synchronisierung des Ressourcenzuriffs mittels Mutex in `main.cpp`

Da für jede Teilaufgabe eine Datei existiert und beide Dateien eine `setup`- und eine `loop`-Funktion enthalten, sollte man beim Kompilieren und Upload darauf achten, dass der Inhalt der nicht benötigten Datei auskommentiert ist.

Demonstration eines Ressourcenkonflikts

Zunächst wurden zwei zusätzliche FreeRTOS-Tasks erstellt, die jeweils Text über die serielle Schnittstelle ausgeben sollen. Dabei besitzt ein Task eine Priorität von 1, der andere eine Priorität von 2. Task 1 gibt dabei seine Nachricht zeichenweise aus, damit nicht der gesamte String im Ganzen zur seriellen Ausgabe kopiert werden kann. Somit kann der String unterbrochen werden. Task 2 gibt seinen String als Ganzes aus, wobei dieser Task aufgrund seiner Priorität von Task 1 unterbrochen werden kann. Um den String von Task 1 mit höherer Wahrscheinlichkeit zu unterbrechen, erfolgt die Zeichenausgabe mit einer Verzögerung von jeweils 10 ms. Infolgedessen werden die beiden Textausgaben ineinander verschachtelt, wodurch der entstehende Ressourcenkonflikt bei gleichzeitigem Zugriff auf die serielle Schnittstelle sichtbar wird.

Die Konsolenausgabe des entstandenen Datenmülls ist hier dargestellt:

```
This is a XXXXXXXXXX
random strXXXXXXXXXX
ing which XXXXXXXXXX
waXXXXXXXXXX
nts to beXXXXXXXXXX
 interruptXXXXXXXXXX
ed!

This isXXXXXXXXXX
 a random XXXXXXXXXX
string whiXXXXXXXXXX
ch wants tXXXXXXXXXX
o be interXXXXXXXXXX
rupted!
```

Synchronisierung mittels Mutex

Zur Behebung des zuvor beobachteten Ressourcenkonflikts wurde ein Mutex implementiert, welcher den Zugriff auf die serielle Schnittstelle synchronisiert. Hierzu wird zunächst mit `xSemaphoreCreateMutex()` ein Mutex erzeugt, auf den beide Tasks zugreifen können. Vor Beginn ihrer jeweiligen Textausgabe versuchen die Tasks, den Mutex mittels `xSemaphoreTake(mutex, portMAX_DELAY)` zu erwerben. Ist dieser bereits durch den jeweils anderen Task belegt, wird der anfordernde Task blockiert, bis der Mutex wieder freigegeben wird. Nach Abschluss der jeweiligen Textausgabe erfolgt die Freigabe durch `xSemaphoreGive(mutex)`, sodass anschließend der andere Task auf die Ressource zugreifen kann. Dadurch wird der Zugriff auf die serielle Schnittstelle synchronisiert, sodass die Tasks jeweils ihren vollständigen String ausgeben können, bevor ein anderer Task auf die serielle Schnittstelle zugreift.

Wie man in der folgenden Konsolenausgabe sehen kann, wurde das Synchronisationsproblem behoben:

```
This is a random string which wants to be interrupted!
XXXXXXXXXX

This is a random string which wants to be interrupted!
XXXXXXXXXX

This is a random string which wants to be interrupted!
XXXXXXXXXX

This is a random string which wants to be interrupted!
XXXXXXXXXX

This is a random string which wants to be interrupted!
XXXXXXXXXX
```

6 Herausforderungen und mögliche Verbesserungen

In diesem Beleg wurde die Umsetzung der Projektaufgaben im Modul *Echtzeitsysteme und mobile Robotik* beschrieben. Neben der erfolgreichen Durchführung des Praktikumsversuchs zum TurtleBot wurde für den bereitgestellten Roboter eine eigene Bewegungssteuerung implementiert. Daraufhin erfolgte die Entwicklung eines Navigationsalgorithmus zur Bewegung des Roboters durch ein Labyrinth. Die Funktionsfähigkeit der implementierten Lösung konnte abschließend demonstriert werden. Darüber hinaus wurden verschiedene Programmieraufgaben zu Echtzeitsystemen bearbeitet und die erfolgreiche Implementierung dokumentiert. Abschließend wurden zwei Zusatzaufgaben mit FreeRTOS umgesetzt.

Im Rahmen der Projektbearbeitung traten verschiedene Herausforderungen auf, für die entsprechende Lösungsansätze entwickelt werden mussten. Darüber hinaus ergeben sich aus den implementierten Lösungen einige Verbesserungsmöglichkeiten. Diese werden in den folgenden Abschnitten erläutert.

Zu Beginn der Entwicklung wurden zunächst alle verfügbaren Sensoren und Aktoren auf ihre Funktionsfähigkeit überprüft. Dabei stellte insbesondere die kontinuierliche Motorsteuerung eine Herausforderung dar. Während der Step-Modus wie erwartet funktionierte, wich das Verhalten des kontinuierlichen Fahrmodus von den Anforderungen ab. Ziel dieser Betriebsart war es, die Geschwindigkeit des Roboters über einen Parameter vorgeben zu können. Dabei sollte ein größerer Parameterwert zu einer höheren Fahrgeschwindigkeit führen. Dieses Verhalten konnte im Rahmen der Funktionstests jedoch nicht bestätigt werden, da kein proportionaler Zusammenhang zwischen dem Parameterwert und der tatsächlichen Geschwindigkeit des Roboters festgestellt werden konnte. In Absprache mit M. Sc. Nico Beyer wurde schließlich festgestellt, dass die kontinuierliche Steuerungsmethode auf dem ATtiny nicht korrekt implementiert ist. Da die Implementierung des ATtiny während der Projektdurchführung nicht geändert werden kann bzw. soll, wurde für die weitere Entwicklung ausschließlich der Step-Modus verwendet. Ein kontinuierlicher Steuerungsmodus hätte insbesondere in Abschnitt 3.1.2 zu einem besseren Ergebnis geführt.

Eine weitere Herausforderung ergab sich bei der Wahl der Kommunikationsart für die Ansteuerung der Aktoren. Ursprünglich war vorgesehen, die Aktoren über ROS 2 Services anzusteuern. Bei der Umsetzung eines einzelnen Aktors als Service-Server und des zugehörigen Service-Clients funktionierte die Kommunikation zunächst wie erwartet. Sobald jedoch mehrere Aktoren auf diese Weise implementiert wurden, trat ein nicht reproduzierbares Verhalten auf. Die Kommunikation funktionierte teilweise, konnte jedoch nicht zuverlässig gewährleistet werden. Als mögliche Ursache können die Ressourcenbegrenzungen des ESP32 genannt werden. Dies konnte allerdings nicht konkret nachgewiesen werden. Da eine zuverlässige und reproduzierbare Ansteuerung der Aktoren eine grundlegende Voraussetzung für den Betrieb des Roboters darstellt, wurde schließlich eine alternative Kommunikationsstruktur auf Basis von Publishern und Subscribern gewählt. Mit dieser Lösung konnte die gewünschte Funktionalität zuverlässig umgesetzt werden, wie in Abschnitt 3.2 beschrieben ist.

Bei den Programmieraufgaben aus den Abschnitten 4.1.4 und 4.2.4 bestehen insbesondere hinsichtlich der Synchronisierung der Peers Verbesserungsmöglichkeiten. Bei der derzeitigen Implementierung werden Änderungen an die anderen Peers nacheinander weitergegeben. Wird beispielsweise eine Nachricht in Peer 1 geändert, übermittelt dieser die Aktualisierung zunächst an Peer 2, anschließend an Peer 3 und so weiter, bis alle ande-

ren Peers im Netzwerk benachrichtigt wurden. Dieses Vorgehen ist vergleichsweise einfach umzusetzen, führt jedoch insbesondere bei einer größeren Anzahl von Peers zu einer längeren Synchronisierungsdauer. Eine effizientere Alternative wäre eine verteilte Weitergabe der Aktualisierungen. Dabei könnten Peers, die eine Änderung bereits erfolgreich empfangen und verarbeitet haben, diese selbstständig an weitere Peers weiterleiten. Für ein Beispiel mit 8 Peers im Netzwerk könnte die Verteilung beispielsweise in mehreren Stufen erfolgen. Zunächst übermittelt Peer 1 die Änderung an Peer 2. Nachdem diese Aktualisierung erfolgreich verarbeitet wurde, können Peer 1 und Peer 2 die Änderung parallel an weitere Peers weitergeben. In einer nächsten Stufe würden somit Peer 1 Peer 3 und Peer 2 Peer 4 benachrichtigen. Anschließend könnten die bereits aktualisierten Peers 1 bis 4 die verbleibenden Peers 5 bis 8 erreichen, indem jeweils ein Peer einem anderen die Änderungen mitteilt und auf die erfolgreiche Synchronisierung wartet. Voraussetzung dafür wäre allerdings eine geeignete Verwaltung der bereits benachrichtigten Peers. Insbesondere müsste sichergestellt werden, dass Aktualisierungen nicht mehrfach verarbeitet oder durch wiederholte Weiterleitungen endlos im Netzwerk verteilt werden. Denkbar wäre hierfür beispielsweise die Verwendung eindeutiger Nachrichtenkennungen sowie eine entsprechende Bestätigung der erfolgreichen Verarbeitung.

Eine weitere Verbesserungsmöglichkeit betrifft die Benutzerinteraktion bei allen beschriebenen Programmieraufgaben. Die bisherige Implementierung beinhaltet Nachrichten mit zufällig bestimmten Inhalten wie beispielsweise Operanden, Zahlenwerte oder Nachrichteninhalte. Eine Erweiterung könnte vorsehen, dass der Anwender eigene Nachrichten über die Kommandozeile eingeben und gezielt vorhandene Chats auswählen kann. Ebenso könnte der Anwender entscheiden, welche Operationen er durchführen möchte. Alternativ wäre die Umsetzung einer grafischen Benutzeroberfläche denkbar, über die sich die Chats und Nachrichten oder Operanden komfortabler auswählen und verwalten lassen.

Literaturverzeichnis

[1] M. Sc. Moritz Thümmler. *Echtzeitsysteme und mobile Robotik*. Praktikumshandout. Nicht veröffentlichte Materialien. 2026.

[2] M. Sc. Nico Beyer. *Echtzeitsysteme und mobile Robotik*. Praktikumsanleitung. Nicht veröffentlichte Materialien. 2026.

[3] *Generate Map - TurtleBot 4 Tutorial*. URL: https://turtlebot.github.io/turtlebot4-user-manual/tutorials/generate_map.html (besucht am 05. 10. 2026).

[4] *Navigation - TurtleBot 4 Tutorial*. URL: https://turtlebot.github.io/turtlebot4-user-manual/tutorials/navigation.html (besucht am 05. 10. 2026).

[5] *Using turtlesim, ros2, and rqt - ROS 2 Tutorial: Humble*. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html (besucht am 07. 10. 2026).

[6] *Creating a ROS 2 Workspace - ROS 2 Documentation: Humble*. URL: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html (besucht am 18. 10. 2026).

[7] *Lösungsalgorithmen für Irrgärten - Pledge-Algorithmus - Wikipedia*. URL: https://de.wikipedia.org/wiki/L%C3%B6sungsalgorithmen_f%C3%BCr_Irrg%C3%A4rten#Pledge-Algorithmus (besucht am 16. 10. 2026).

[8] M. Sc. Moritz Thümmler. *Echtzeitsysteme und mobile Robotik*. Programmieraufgaben. Nicht veröffentlichte Materialien. 2026.

[9] M. Sc. Moritz Thümmler. *Echtzeitsysteme und mobile Robotik*. Beispieldateien. Nicht veröffentlichte Materialien. 2026.

[10] *DigiKey - Introduction to RTOS*. URL: https://www.youtube.com/playlist?list=PLEBQazB0HUyQ4hAPU1cJED6t3DU0h34bz (besucht am 29. 10. 2026).